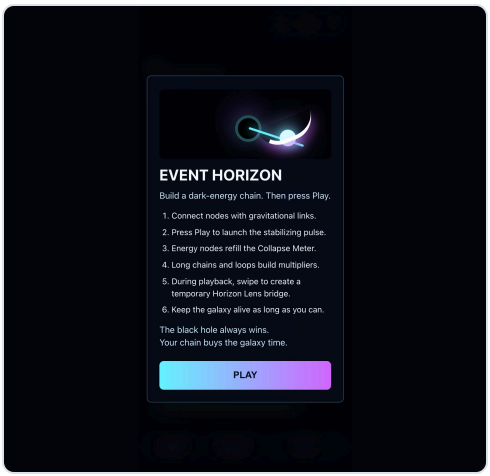
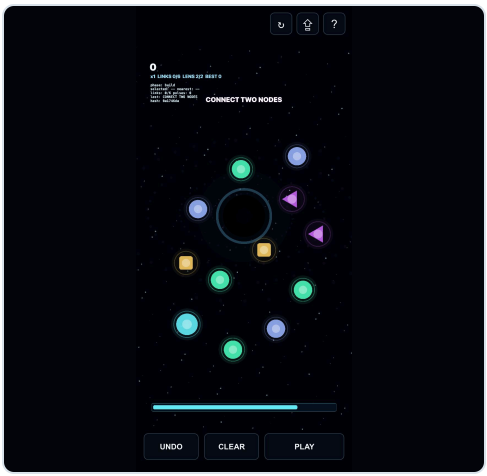
Event Horizon Iteration 03 Report

Pulse Chain pivot: planning, payoff, and Horizon Lens rescue play.

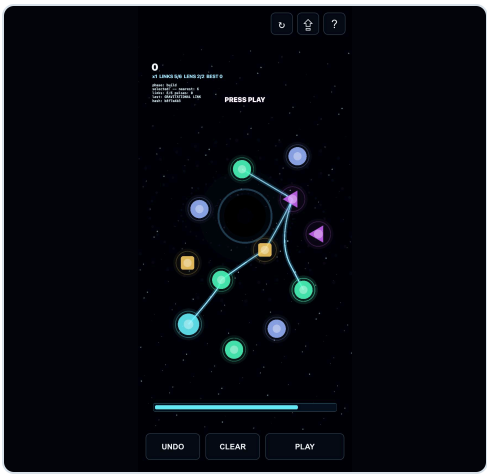
Screenshots



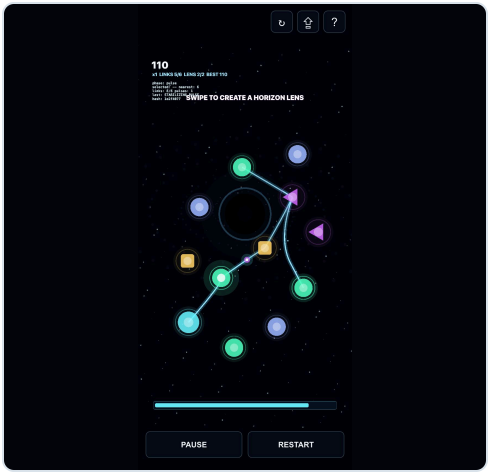
Help overlay



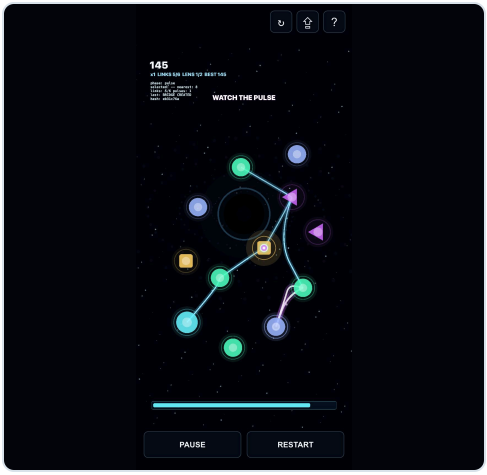
Build phase



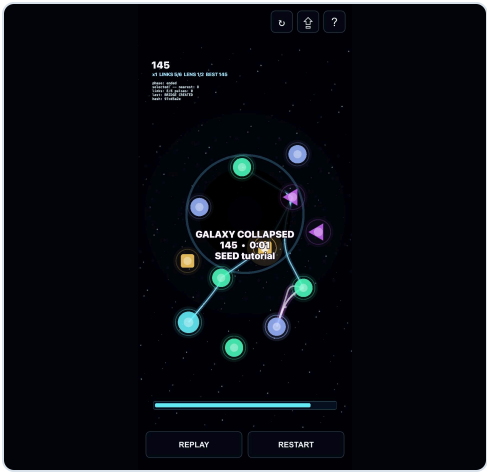
Link placement



Pulse running



Horizon Lens bridge



End screen

Changed Source Index

- README.md
- index.html
- netlify/functions/score-submit.mjs
- package.json
- scripts/capture-iteration-03-artifacts.mjs
- scripts/generate-iteration-03-report.mjs
- scripts/test-score-submit.mjs
- src/game/pulse/PulseGeometry.ts
- src/game/pulse/PulseInputController.ts
- src/game/pulse/PulseLevelGenerator.ts
- src/game/pulse/PulseMode.ts
- src/game/pulse/PulseRenderer.ts
- src/game/pulse/PulseSimulation.ts
- src/game/pulse/PulseTypes.ts
- src/game/scoreClient.ts
- src/main.ts
- src/styles.css
- tests/e2e/playable.spec.ts
- tests/pulse-simulation.test.ts
- tests/score-submit.test.ts

Event Horizon Iteration 03 Report

Summary Of Pivot

Iteration 03 pivots Event Horizon from tether-swiping survival into the default Pulse Chain mode: connect Dark Energy Nodes with Gravitational Links, press Play, watch a Stabilizing Pulse traverse the network, and rescue the run with short-lived Horizon Lens bridges during playback.

Why The Old Loop Was Not Fun Enough

The earlier prototype improved mobile input, but its main action still felt like touching a moving target. It asked for precision during chaos before the player understood the plan. Pulse Chain moves the fun toward planning, payoff, readable cause and effect, and one live skill action that supports the puzzle instead of becoming the entire challenge.

New Gameplay Explanation

- Build phase: tap-tap or drag from one node to another to place a directional Gravitational Link.
- Pulse phase: press Play to launch a Stabilizing Pulse from the Source Node.
- Scoring: Energy Nodes refill the Collapse Meter, long chains raise multipliers, Splitter Nodes branch pulses, and Delay Nodes hold timing.
- Rescue phase: swipe during playback to draw a Horizon Lens. If the swipe anchors near two valid nodes, it creates a temporary bridge.
- End state: reaching the score or survival target stabilizes the sector; empty energy collapses the galaxy.

Exact Files Changed

- README.md
- index.html
- netlify/functions/score-submit.mjs
- package.json
- scripts/capture-iteration-03-artifacts.mjs
- scripts/generate-iteration-03-report.mjs
- scripts/test-score-submit.mjs
- src/game/pulse/PulseGeometry.ts
- src/game/pulse/PulseInputController.ts
- src/game/pulse/PulseLevelGenerator.ts
- src/game/pulse/PulseMode.ts
- src/game/pulse/PulseRenderer.ts
- src/game/pulse/PulseSimulation.ts
- src/game/pulse/PulseTypes.ts
- src/game/scoreClient.ts
- src/main.ts
- src/styles.css
- tests/e2e/playable.spec.ts
- tests/pulse-simulation.test.ts
- tests/score-submit.test.ts
- docs/artifacts/iteration-03-build-phase-mobile.jpg
- docs/artifacts/iteration-03-end-screen-mobile.jpg
- docs/artifacts/iteration-03-help-mobile.jpg
- docs/artifacts/iteration-03-horizon-lens-mobile.jpg
- docs/artifacts/iteration-03-link-placement-mobile.jpg
- docs/artifacts/iteration-03-pulse-running-mobile.jpg
- docs/artifacts/iteration-03-test-results.txt

Diff Summary

```
README.md                          |  70 ++++++++---
 index.html                        |  20 ++--
 netlify/functions/score-submit.mjs |  22 +++-
 package.json                      |   4 +-
 scripts/test-score-submit.mjs     |  63 ++++++++--
 src/game/scoreClient.ts           |   3 +-
 src/main.ts                       | 230 ++++++++++++++++++++++++++++++------
 src/styles.css                    |  51 +++++++-
 tests/e2e/playable.spec.ts        | 216 +++++++++++++++++++++++++++++++----
 tests/score-submit.test.ts        |  47 ++++++++
 10 files changed, 554 insertions(+), 172 deletions(-)
```

Tests Run And Results

```
COMMAND: npm run build

> event-horizon@0.1.0 build
> tsc --noEmit && vite build

vite v8.0.14 building client environment for production...
 [2K
transforming...✓ 724 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html                                 3.36 kB │ gzip:  1.15 kB
dist/assets/index-C0riv6bZ.css                  4.27 kB │ gzip:  1.50 kB
dist/assets/webworkerAll-u0-LKNcn.js            0.06 kB │ gzip:  0.07 kB
dist/assets/WebGLRenderer-Dr5I3uql.js           0.07 kB │ gzip:  0.07 kB
dist/assets/CanvasRenderer-C_VE2uLM.js          0.07 kB │ gzip:  0.08 kB
dist/assets/WebGPURenderer-BqF0A8r4.js          0.07 kB │ gzip:  0.08 kB
dist/assets/init-nWfKNZF3.js                    0.16 kB │ gzip:  0.16 kB │ map:   0.57 kB
dist/assets/browserAll-DjO1SdvZ.js              0.29 kB │ gzip:  0.23 kB │ map:   1.51 kB
dist/assets/getTextureBatchBindGroup-Dw2p-Wfk.js    0.40 kB │ gzip:  0.31 kB │ map:   1.78 kB
dist/assets/CanvasPool-9Gf99FtZ.js              0.80 kB │ gzip:  0.45 kB │ map:   3.50 kB
dist/assets/canvasUtils-By6x9n4d.js             6.07 kB │ gzip:  2.07 kB │ map:  21.84 kB
dist/assets/BufferResource-CHMjz1hG.js         10.57 kB │ gzip:  2.81 kB │ map:  25.57 kB
dist/assets/WebGPURenderer-CaI5dhXK.js         37.78 kB │ gzip: 10.56 kB │ map: 121.24 kB
dist/assets/FilterSystem-DKkk6B7X.js           40.30 kB │ gzip: 13.07 kB │ map: 159.86 kB
dist/assets/FederatedEventTarget-BGQ8UmS3.js   42.56 kB │ gzip: 11.09 kB │ map: 155.08 kB
dist/assets/WebGLRenderer-4UnqJObN.js          67.75 kB │ gzip: 18.59 kB │ map: 214.44 kB
dist/assets/RenderTargetSystem-D4_io0eT.js     77.17 kB │ gzip: 22.67 kB │ map: 295.31 kB
dist/assets/CanvasRenderer-lu0yL7Nx.js         79.94 kB │ gzip: 24.36 kB │ map: 404.18 kB
dist/assets/Geometry-CiJBQvr3.js              101.60 kB │ gzip: 31.13 kB │ map: 498.88 kB
dist/assets/index-hTwEF4Vy.js                 111.50 kB │ gzip: 34.29 kB │ map: 469.31 kB

✓ built in 223ms

COMMAND: npm run lint

> event-horizon@0.1.0 lint
> eslint .


COMMAND: npm run test
```

```
> event-horizon@0.1.0 test
> vitest run


 RUN  v4.0.15 /Users/kevinhegg/Desktop/event-horizon

 ✓ tests/score-submit.test.ts (3 tests) 15ms
 ✓ tests/simulation.test.ts (7 tests) 9ms
 ✓ tests/pulse-simulation.test.ts (10 tests) 9ms

 Test Files  3 passed (3)
      Tests  20 passed (20)
   Start at  12:53:24
   Duration  187ms (transform 142ms, setup 0ms, import 184ms, tests 33ms, environment 0ms)


COMMAND: npm run test:e2e

> event-horizon@0.1.0 test:e2e
> playwright test


Running 6 tests using 1 worker

(node:58301) Warning: The 'NO_COLOR' env is ignored due to the 'FORCE_COLOR' env being set.
(Use `node --trace-warnings ...` to show where the warning was created)
(node:58301) Warning: The 'NO_COLOR' env is ignored due to the 'FORCE_COLOR' env being set.
(Use `node --trace-warnings ...` to show where the warning was created)
  ✓  1 [mobile-chrome] › tests/e2e/playable.spec.ts:4:1 › help opens on first visit (404ms)
  ✓  2 [mobile-chrome] › tests/e2e/playable.spec.ts:12:1 › connect two nodes by tap-tap (357ms)
  ✓  3 [mobile-chrome] › tests/e2e/playable.spec.ts:29:1 › connect two nodes by drag (506ms)
  ✓  4 [mobile-chrome] › tests/e2e/playable.spec.ts:52:1 › press play, pulse moves, and energy node scores (1.1s)
  ✓  5 [mobile-chrome] › tests/e2e/playable.spec.ts:80:1 › swipe during pulse phase creates Horizon Lens and records replay inputs (844ms)
  ✓  6 [mobile-chrome] › tests/e2e/playable.spec.ts:119:1 › collapse or stabilized end state is reachable (461ms)

  6 passed (4.5s)

COMMAND: npm run score:test

> event-horizon@0.1.0 score:test
> node scripts/test-score-submit.mjs

[
  {
    "status": 200,
    "body": {
      "ok": true,
      "acceptedAt": "2026-05-30T16:53:29.520Z",
      "score": 1280,
      "survivalMs": 67421
    }
  },
  {
    "status": 200,
    "body": {
      "ok": true,
      "acceptedAt": "2026-05-30T16:53:29.525Z",
      "score": 620,
      "survivalMs": 12120
    }
  }
]

COMMAND: npm run capture:iteration-03

> event-horizon@0.1.0 capture:iteration-03
> node scripts/capture-iteration-03-artifacts.mjs

Captured docs/artifacts/iteration-03-help-mobile.jpg
Captured docs/artifacts/iteration-03-build-phase-mobile.jpg
Captured docs/artifacts/iteration-03-link-placement-mobile.jpg
Captured docs/artifacts/iteration-03-pulse-running-mobile.jpg
Captured docs/artifacts/iteration-03-horizon-lens-mobile.jpg
Captured docs/artifacts/iteration-03-end-screen-mobile.jpg

COMMAND: npm run report:iteration-03

> event-horizon@0.1.0 report:iteration-03
> node scripts/generate-iteration-03-report.mjs

Report generated after this log was written.
```

Screenshots

- docs/artifacts/iteration-03-help-mobile.jpg
- docs/artifacts/iteration-03-build-phase-mobile.jpg
- docs/artifacts/iteration-03-link-placement-mobile.jpg
- docs/artifacts/iteration-03-pulse-running-mobile.jpg
- docs/artifacts/iteration-03-horizon-lens-mobile.jpg
- docs/artifacts/iteration-03-end-screen-mobile.jpg

Known Limitations

- Playwright mobile simulation passed; this run did not include hands-on testing on a physical phone.
- Horizon Lens uses the understandable temporary-link mechanic for this iteration, not full pulse deflection.
- When all pulses die, the run currently ends with accelerated collapse pressure represented as a pulse-died end reason; future tuning should return players to build/retry more gracefully.
- Node labels are visualized mostly through shape and color; richer in-canvas labels and sound are good next steps.
- GitHub Pages serves the static game only. Netlify Functions remain available only on Netlify or local Netlify dev.

Next Recommended Iteration

- Tune the tutorial seed on a real iPhone and Android phone.
- Add sound, haptics, and stronger pulse arrival bursts.
- Add a retry flow that preserves the same seed and highlights the failed dead end.
- Build a replay viewer for the Pulse Chain payload.
- Add leaderboards or local seed challenge sharing once the loop feels sticky.

Full Diffs For Tracked Changes

```
diff --git a/README.md b/README.md
index db525cf..fdd1cfc 100644
--- a/README.md
+++ b/README.md
@@ -1,5 +1,5 @@
 # Event Horizon

-Event Horizon is a mobile-first, one-thumb casual survival prototype about delaying a galaxy's collapse into a black hole. The first vertical slice is a plain Vite + PixiJS v8 site with deterministic simulation state kept outside the renderer.
+Event Horizon is a mobile-first cosmic chain-reaction game about delaying a galaxy's collapse into a black hole. The current slice is a plain Vite + PixiJS v8 site with deterministic simulation state kept outside the renderer.

 ## Current Slice
@@ -9,16 +9,19 @@ Event Horizon is a mobile-first, one-thumb casual survival prototype about delay
 - Fixed `60 Hz` simulation step decoupled from render frames
 - Seeded `mulberry32` RNG and replay payloads from seed + input timings
-- Path-based tap/swipe input recording with mobile Pointer Events and TouchEvent fallback
-- Dark-energy orbs, tether-based swipe capture, flyby bonuses, and shadow-arm hazards
-- First-run help overlay, tutorial ramp, visible swipe trails, hit/miss feedback, and debug hooks
-- Phase-based black-hole visibility and gravity pressure
-- Bottom dark-energy meter, score/time HUD, and collapse end animation
+- Default Pulse Chain mode: connect Dark Energy Nodes, press Play, and watch a Stabilizing Pulse travel the network
+- Tap-tap and drag link placement with a visible link budget, undo, clear, and reset
+- Energy, Delay, Splitter, Conduit, and Source nodes with scoring and multiplier rules
+- Horizon Lens swipes during pulse playback create short-lived temporary bridges
+- Path-based input recording with mobile Pointer Events and TouchEvent fallback
+- First-run help overlay, tutorial hints, visible pulse/lens feedback, and debug hooks
+- Bottom Collapse Meter, score/multiplier HUD, and stabilized/collapsed end states
 - Share poster export from three captured gameplay frames
-- Minimal Netlify score submit function plus Google Apps Script sample
+- Minimal Netlify score submit function that accepts legacy and Pulse Chain replay payloads
+- Previous tether survival prototype remains available with `?mode=legacy`

 ## Assumptions

 - The initial repo was empty, so this is a greenfield Vite scaffold.
-- Custom deterministic motion is enough for the vertical slice; no Matter.js or Planck.js dependency is needed yet.
+- Custom deterministic motion is enough for the Pulse Chain slice; no Matter.js or Planck.js dependency is needed.
 - GitHub Pages is the public static target and uses `/event-horizon/` as the base path.
 - Netlify deploys should set `VITE_BASE_PATH=/` through the included `netlify.toml`.
@@ -35,4 +38,6 @@ npm run test
 npm run test:e2e
 npm run score:test
+npm run capture:iteration-03
+npm run report:iteration-03
 npm run capture:iteration-02
 npm run report:iteration-02
@@ -83,4 +88,43 @@ npx netlify dev
 ## Replay Payload Shape

+```json
+{
+  "version": 1,
+  "mode": "pulse-chain",
+  "seed": "tutorial",
+  "startedAt": 1780185600000,
+  "buildInputs": [
+    { "t": 0, "kind": "link", "fromId": 1, "toId": 2 },
+    { "t": 300, "kind": "link", "fromId": 2, "toId": 3 },
+    { "t": 620, "kind": "play" }
+  ],
+  "liveInputs": [
+    {
+      "t": 1600,
+      "kind": "lens",
+      "path": [
+        { "x": 835, "y": 1215, "t": 0 },
+        { "x": 700, "y": 1410, "t": 120 }
+      ],
+      "fromId": 6,
+      "toId": 8,
+      "success": true
+    }
+  ],
+  "result": {
+    "score": 620,
+    "survivalMs": 12120,
+    "maxMultiplier": 2,
+    "loopsCompleted": 1,
+    "linksUsed": 5,
+    "stabilized": false,
+    "collapsed": false
+  },
+  "stepHash": "04ce9b1a"
+}
+```
+
+Legacy mode still uses:
+
 ```json
 {
@@ -98,11 +142,11 @@ npx netlify dev
 ```

-## First PR Workflow
+## Current Branch Workflow

 ```bash
-git switch -c feat/first-playable
+git switch -c feat/pulse-chain-pivot
 git add .
-git commit -m "Build first playable Event Horizon slice"
-git push -u origin feat/first-playable
-gh pr create --base main --head feat/first-playable --title "Build first playable Event Horizon slice" --body-file docs/pr-body.md
+git commit -m "Pivot Event Horizon to pulse chain gameplay"
+git push -u origin feat/pulse-chain-pivot
+gh pr create --base main --head feat/pulse-chain-pivot --title "Pivot Event Horizon to pulse chain gameplay" --body-file docs/iteration-03-report.md
 ```
diff --git a/index.html b/index.html
index 7de8c73..ba08306 100644
--- a/index.html
+++ b/index.html
@@ -18,4 +18,9 @@
           ⇩
         </div>
+   <div id="pulse-controls" aria-live="polite">
+   <button id="pulse-undo-button" type="button">Undo</button>
+   <button id="pulse-clear-button" type="button">Clear</button>
+   <button id="pulse-play-button" type="button">Play</button>
+   </div>
         <section id="help-overlay" aria-modal="true" role="dialog" aria-labelledby="help-title" hidden>
           <div id="help-panel">
@@ -27,13 +32,14 @@
             </div>
             <h1 id="help-title">EVENT HORIZON</h1>
-            <p>Hold back the collapse of the galaxy.</p>
+   <p>Build a dark-energy chain. Then press Play.</p>

-              Swipe through glowing tethers to harvest dark-energy orbs.
```

```
-              <li>Tap an orb to stabilize it briefly.</li>
-              <li>Tap streaking stars for a bonus.</li>
-              <li>Keep the Dark Energy meter alive.</li>
-              <li>When the meter empties, the galaxy collapses.</li>
+              <li>Connect nodes with gravitational links.</li>
+              <li>Press Play to launch the stabilizing pulse.</li>
+              <li>Energy nodes refill the Collapse Meter.</li>
+              <li>Long chains and loops build multipliers.</li>
+              <li>During playback, swipe to create a temporary Horizon Lens bridge.</li>
+              <li>Keep the galaxy alive as long as you can.</li>
             </ol>
-            <p>The black hole always wins.<br />Your job is to delay it.</p>
+            <p>The black hole always wins.<br />Your chain buys the galaxy time.</p>
             <button id="help-play-button" type="button">PLAY</button>
           </div>
diff --git a/netlify/functions/score-submit.mjs b/netlify/functions/score-submit.mjs
index 982d23f..881ec83 100644
--- a/netlify/functions/score-submit.mjs
+++ b/netlify/functions/score-submit.mjs
@@ -30,6 +30,6 @@ export default async function scoreSubmit(request) {
     ok: true,
     acceptedAt: new Date().toISOString(),
-    score: payload.score,
-    survivalMs: payload.survivalMs
+    score: payload.mode === 'pulse-chain' ? payload.result.score : payload.score,
+    survivalMs: payload.mode === 'pulse-chain' ? payload.result.survivalMs : payload.survivalMs
   });
 }
@@ -49,4 +49,22 @@ function validateReplay(payload) {
     return 'version_required';
   }
+  if (payload.mode === 'pulse-chain') {
+    if (typeof payload.seed !== 'string' || payload.seed.length < 3) {
+      return 'seed_required';
+    }
+    if (!Number.isFinite(payload.startedAt)) {
+      return 'started_at_required';
+    }
+    if (!Array.isArray(payload.buildInputs) || !Array.isArray(payload.liveInputs)) {
+      return 'inputs_required';
+    }
+    if (!payload.result || !Number.isFinite(payload.result.score) || !Number.isFinite(payload.result.survivalMs)) {
+      return 'result_required';
+    }
+    if (typeof payload.stepHash !== 'string') {
+      return 'hash_required';
+    }
+    return '';
+  }
   if (typeof payload.seed !== 'string' || payload.seed.length < 3) {
     return 'seed_required';
diff --git a/package.json b/package.json
index 9f8bec1..dee2931 100644
--- a/package.json
+++ b/package.json
@@ -14,6 +14,8 @@
     "capture:artifacts": "node scripts/capture-artifacts.mjs",
     "capture:iteration-02": "node scripts/capture-iteration-02-artifacts.mjs",
+    "capture:iteration-03": "node scripts/capture-iteration-03-artifacts.mjs",
     "report:pdf": "node scripts/generate-report-pdf.mjs",
-    "report:iteration-02": "node scripts/generate-iteration-02-report.mjs"
+    "report:iteration-02": "node scripts/generate-iteration-02-report.mjs",
+    "report:iteration-03": "node scripts/generate-iteration-03-report.mjs"
   },
   "dependencies": {
diff --git a/scripts/test-score-submit.mjs b/scripts/test-score-submit.mjs
index 4ba1ba0..6d1bb1c 100644
--- a/scripts/test-score-submit.mjs
+++ b/scripts/test-score-submit.mjs
@@ -14,17 +14,56 @@ const replay = {
 };

-const response = await scoreSubmit(
-  new Request('https://event-horizon.test/.netlify/functions/score-submit', {
-    method: 'POST',
-    headers: { 'content-type': 'application/json' },
-    body: JSON.stringify(replay)
-  })
-);
+const pulseReplay = {
+  version: 1,
+  mode: 'pulse-chain',
+  seed: 'tutorial',
+  startedAt: 1780185600000,
+  buildInputs: [
+    { t: 0, kind: 'link', fromId: 1, toId: 2 },
+    { t: 300, kind: 'link', fromId: 2, toId: 3 },
+    { t: 620, kind: 'play' }
+  ],
+  liveInputs: [
+    {
+      t: 1600,
+      kind: 'lens',
+      path: [
+        { x: 835, y: 1215, t: 0 },
+        { x: 700, y: 1410, t: 120 }
+      ],
+      fromId: 6,
+      toId: 8,
+      success: true
+    }
+  ],
+  result: {
+    score: 620,
+    survivalMs: 12120,
+    maxMultiplier: 2,
+    loopsCompleted: 1,
+    linksUsed: 5,
+    stabilized: false,
+    collapsed: false
+  },
+  stepHash: '04ce9b1a'
+};

-const body = await response.json();
-if (response.status !== 200 || !body.ok || body.score !== replay.score) {
-  console.error(JSON.stringify({ status: response.status, body }, null, 2));
-  process.exit(1);
+const results = [];
+for (const payload of [replay, pulseReplay]) {
+  const response = await scoreSubmit(
+    new Request('https://event-horizon.test/.netlify/functions/score-submit', {
+      method: 'POST',
+      headers: { 'content-type': 'application/json' },
+      body: JSON.stringify(payload)
+    })
+  );
+  const body = await response.json();
+  const expectedScore = payload.mode === 'pulse-chain' ? payload.result.score : payload.score;
+  if (response.status !== 200 || !body.ok || body.score !== expectedScore) {
+    console.error(JSON.stringify({ status: response.status, body }, null, 2));
+    process.exit(1);
+  }
```

```
+  results.push({ status: response.status, body });
 }

-console.log(JSON.stringify({ status: response.status, body }, null, 2));
+console.log(JSON.stringify(results, null, 2));
diff --git a/src/game/scoreClient.ts b/src/game/scoreClient.ts
index aa47e28..7c2df53 100644
--- a/src/game/scoreClient.ts
+++ b/src/game/scoreClient.ts
@@ -1,3 +1,4 @@
 import { SCORE_ENDPOINT } from './constants';
+import type { PulseReplayPayload } from './pulse/PulseTypes';
 import type { ReplayPayload } from './types';

@@ -9,5 +10,5 @@ export interface ScoreSubmitResult {

 export async function submitScore(
-  payload: ReplayPayload,
+  payload: ReplayPayload | PulseReplayPayload,
   endpoint = SCORE_ENDPOINT
 ): Promise<ScoreSubmitResult> {
diff --git a/src/main.ts b/src/main.ts
index 73ffd29..c0c1ef1 100644
--- a/src/main.ts
+++ b/src/main.ts
@@ -1,4 +1,18 @@
 import './styles.css';
 import { EventHorizonGame } from './game/EventHorizonGame';
+import { PulseMode } from './game/pulse/PulseMode';
+import { getDailyPulseSeed } from './game/pulse/PulseLevelGenerator';
+
+interface EventHorizonRuntime {
+  start: () => Promise<void>;
+  restart: () => void;
+  setPaused: (paused: boolean) => void;
+  setInputDebug: (enabled: boolean) => void;
+  exportPoster: () => Promise<string>;
+  forceEnd: () => void;
+  getSnapshot: () => unknown;
+  getReplayPayload: () => unknown;
+  destroy?: () => void;
+}

 const root = document.querySelector<HTMLDivElement>('#game-root');
@@ -9,22 +23,51 @@ const helpOverlay = document.querySelector<HTMLElement>('#help-overlay');
 const helpPlayButton = document.querySelector<HTMLButtonElement>('#help-play-button');
 const posterLink = document.querySelector<HTMLAnchorElement>('#poster-link');
+const pulseControls = document.querySelector<HTMLElement>('#pulse-controls');
+const pulseUndoButton = document.querySelector<HTMLButtonElement>('#pulse-undo-button');
+const pulseClearButton = document.querySelector<HTMLButtonElement>('#pulse-clear-button');
+const pulsePlayButton = document.querySelector<HTMLButtonElement>('#pulse-play-button');

-if (!root || !restartButton || !shareButton || !helpButton || !helpOverlay || !helpPlayButton || !posterLink) {
+if (
+  !root ||
+  !restartButton ||
+  !shareButton ||
+  !helpButton ||
+  !helpOverlay ||
+  !helpPlayButton ||
+  !posterLink ||
+  !pulseControls ||
+  !pulseUndoButton ||
+  !pulseClearButton ||
+  !pulsePlayButton
+) {
   throw new Error('Event Horizon shell is missing required DOM nodes.');
 }

-const game = new EventHorizonGame(root, {
-  seed: 'eh-2026-05-29-alpha',
-  startedAt: 1780051200000
-});
+const params = new URLSearchParams(window.location.search);
+const mode = params.get('mode') === 'legacy' ? 'legacy' : 'pulse-chain';
+const debugInput = params.get('debugInput') === '1';
+const seed = params.get('seed') ?? getDailyPulseSeed();

-await game.start();
+const game: EventHorizonRuntime =
+  mode === 'legacy'
+    ? new EventHorizonGame(root, {
+        seed: 'eh-2026-05-29-alpha',
+        startedAt: 1780051200000
+      })
+    : new PulseMode(root, {
+        seed,
+        startedAt: Date.now()
+      });

-const debugInput = new URLSearchParams(window.location.search).get('debugInput') === '1';
+await game.start();
 game.setInputDebug(debugInput);
+pulseControls.hidden = mode === 'legacy';
+let pulsePaused = false;
+
+const helpKey = mode === 'legacy' ? 'eventHorizon.helpSeen' : 'eventHorizon.pulseHelpSeen';

 const hasSeenHelp = (): boolean => {
   try {
-    return localStorage.getItem('eventHorizon.helpSeen') === '1';
+    return localStorage.getItem(helpKey) === '1';
   } catch {
     return false;
@@ -34,5 +77,5 @@ const hasSeenHelp = (): boolean => {
 const markHelpSeen = (): void => {
   try {
-    localStorage.setItem('eventHorizon.helpSeen', '1');
+    localStorage.setItem(helpKey, '1');
   } catch {
     // localStorage can be unavailable in locked-down browser modes.
@@ -42,5 +85,5 @@ const markHelpSeen = (): void => {
 const openHelp = (): void => {
   helpOverlay.hidden = false;
-  helpPlayButton.textContent = game.getSnapshot().timeMs > 0 ? 'RESUME' : 'PLAY';
+  helpPlayButton.textContent = 'PLAY';
   game.setPaused(true);
 };
@@ -58,5 +101,7 @@ if (!hasSeenHelp()) {
 restartButton.addEventListener('click', () => {
   posterLink.removeAttribute('href');
+  pulsePaused = false;
   game.restart();
+  updatePulseControls();
 });

@@ -69,4 +114,81 @@ shareButton.addEventListener('click', async () => {
 });

+if (game instanceof PulseMode) {
+  pulseUndoButton.addEventListener('click', () => {
+    const snapshot = game.getSnapshot();
+    if (snapshot.phase === 'build') {
```

```
+      game.undo();
+    } else if (snapshot.phase === 'pulse') {
+      pulsePaused = !pulsePaused;
+      game.setPaused(pulsePaused);
+    } else {
+      pulsePaused = false;
+      game.restart();
+    }
+    updatePulseControls();
+  });
+  pulseClearButton.addEventListener('click', () => {
+    const snapshot = game.getSnapshot();
+    if (snapshot.phase === 'build') {
+      game.clearLinks();
+    } else {
+      pulsePaused = false;
+      game.restart();
+    }
+    updatePulseControls();
+  });
+  pulsePlayButton.addEventListener('click', () => {
+    if (game.getSnapshot().phase === 'build') {
+      game.playPulse();
+      pulsePaused = false;
+      updatePulseControls();
+    }
+  });
+  window.setInterval(updatePulseControls, 250);
+  updatePulseControls();
+} else {
+  pulseControls.hidden = true;
+}
+
+function updatePulseControls(): void {
+  if (!(game instanceof PulseMode)) {
+    return;
+  }
+  const controls = pulseControls;
+  const undoButton = pulseUndoButton;
+  const clearButton = pulseClearButton;
+  const playButton = pulsePlayButton;
+  if (!controls || !undoButton || !clearButton || !playButton) {
+    return;
+  }
+  const snapshot = game.getSnapshot();
+  controls.dataset.phase = snapshot.phase;
+  if (snapshot.phase === 'build') {
+    undoButton.hidden = false;
+    clearButton.hidden = false;
+    playButton.hidden = false;
+    undoButton.textContent = 'Undo';
+    clearButton.textContent = 'Clear';
+    playButton.textContent = 'Play';
+    playButton.disabled = snapshot.linksUsed === 0;
+    return;
+  }
+  if (snapshot.phase === 'pulse') {
+    undoButton.hidden = false;
+    clearButton.hidden = false;
+    playButton.hidden = true;
+    undoButton.textContent = pulsePaused ? 'Resume' : 'Pause';
+    clearButton.textContent = 'Restart';
+    playButton.disabled = true;
+    return;
+  }
+  undoButton.hidden = false;
+  clearButton.hidden = false;
+  playButton.hidden = true;
+  undoButton.textContent = 'Replay';
+  clearButton.textContent = 'Restart';
+  playButton.disabled = true;
+}
+
 declare global {
   interface Window {
@@ -74,16 +196,25 @@ declare global {
       exportPoster: () => Promise<string>;
       forceEnd: () => void;
-      getReplayPayload: () => ReturnType<EventHorizonGame['getReplayPayload']>;
-      getSnapshot: () => ReturnType<EventHorizonGame['getSnapshot']>;
+      getReplayPayload: () => unknown;
+      getSnapshot: () => unknown;
       restart: () => void;
     };
     __EVENT_HORIZON_DEBUG__?: {
+      addLink: (fromId: number, toId: number) => unknown;
+      clearLinks: () => unknown;
+      forceBuildPhase: () => void;
+      forceCollapse: () => void;
       forceHelp: (open: boolean) => void;
-      getLastGesture: () => ReturnType<EventHorizonGame['getLastGesture']>;
-      getReplayPayload: () => ReturnType<EventHorizonGame['getReplayPayload']>;
-      getSnapshot: () => ReturnType<EventHorizonGame['getSnapshot']>;
-      forcePulsePhase: () => void;
+      getLastInputResult: () => unknown;
+      getLinks: () => unknown;
+      getMode: () => string;
+      getNodes: () => unknown;
+      getPulses: () => unknown;
+      getReplayPayload: () => unknown;
+      getSnapshot: () => unknown;
+      playPulse: () => unknown;
       setInputDebug: (enabled: boolean) => void;
-      simulateSwipeWorld: (points: { x: number; y: number }[]) => ReturnType<EventHorizonGame['simulateSwipeWorld']>;
-      simulateTapWorld: (x: number, y: number) => ReturnType<EventHorizonGame['simulateTapWorld']>;
+      simulateLens: (points: { x: number; y: number }[]) => unknown;
     };
   }
@@ -98,17 +229,52 @@ window.__EVENT_HORIZON__ = {
 };

-window.__EVENT_HORIZON_DEBUG__ = {
-  forceHelp: (open) => {
-    if (open) {
-      openHelp();
-    } else {
-      closeHelp();
-    }
-  },
-  getLastGesture: () => game.getLastGesture(),
-  getReplayPayload: () => game.getReplayPayload(),
-  getSnapshot: () => game.getSnapshot(),
-  setInputDebug: (enabled) => game.setInputDebug(enabled),
-  simulateSwipeWorld: (points) => game.simulateSwipeWorld(points),
-  simulateTapWorld: (x, y) => game.simulateTapWorld(x, y)
-};
+window.__EVENT_HORIZON_DEBUG__ =
+  game instanceof PulseMode
+    ? {
+        addLink: (fromId, toId) => game.addLink(fromId, toId),
+        clearLinks: () => game.clearLinks(),
+        forceBuildPhase: () => game.forceBuildPhase(),
+        forceCollapse: () => game.forceCollapse(),
```

```
+        forceHelp: (open) => {
+          if (open) {
+            openHelp();
+          } else {
+            closeHelp();
+          }
+        },
+        forcePulsePhase: () => game.forcePulsePhase(),
+        getLastInputResult: () => game.getLastInputResult(),
+        getLinks: () => game.getLinks(),
+        getMode: () => game.getMode(),
+        getNodes: () => game.getNodes(),
+        getPulses: () => game.getPulses(),
+        getReplayPayload: () => game.getReplayPayload(),
+        getSnapshot: () => game.getSnapshot(),
+        playPulse: () => game.playPulse(),
+        setInputDebug: (enabled) => game.setInputDebug(enabled),
+        simulateLens: (points) => game.simulateLens(points)
+      }
+    : {
+        addLink: () => undefined,
+        clearLinks: () => undefined,
+        forceBuildPhase: () => undefined,
+        forceCollapse: () => game.forceEnd(),
+        forceHelp: (open) => {
+          if (open) {
+            openHelp();
+          } else {
+            closeHelp();
+          }
+        },
+        forcePulsePhase: () => undefined,
+        getLastInputResult: () => undefined,
+        getLinks: () => [],
+        getMode: () => 'legacy',
+        getNodes: () => [],
+        getPulses: () => [],
+        getReplayPayload: () => game.getReplayPayload(),
+        getSnapshot: () => game.getSnapshot(),
+        playPulse: () => undefined,
+        setInputDebug: (enabled) => game.setInputDebug(enabled),
+        simulateLens: () => undefined
+      };
diff --git a/src/styles.css b/src/styles.css
index 20a7fa1..7413c0f 100644
--- a/src/styles.css
+++ b/src/styles.css
@@ -77,11 +77,31 @@ a {
 }

+#pulse-controls {
+  position: fixed;
+  right: max(14px, env(safe-area-inset-right));
+  bottom: max(22px, env(safe-area-inset-bottom));
+  left: max(14px, env(safe-area-inset-left));
+  z-index: 3;
+  display: grid;
+  grid-template-columns: 1fr 1fr 1.45fr;
+  gap: 10px;
+  pointer-events: auto;
+}
+
+#pulse-controls[hidden] {
+  display: none;
+}
+
+#pulse-controls[data-phase="pulse"],
+#pulse-controls[data-phase="ended"] {
+  grid-template-columns: 1fr 1fr;
+}
+
 #restart-button,
 #share-button,
 #help-button,
-#poster-link {
+#poster-link,
+#pulse-controls button {
   display: grid;
-  width: 40px;
-  height: 40px;
   place-items: center;
   border: 1px solid rgba(179, 226, 255, 0.26);
@@ -96,4 +116,29 @@ a {
 }

+#restart-button,
+#share-button,
+#help-button,
+#poster-link {
+  width: 40px;
+  height: 40px;
+}
+
+#pulse-controls button {
+  min-height: 48px;
+  color: #f7fbff;
+  font-size: 14px;
+  font-weight: 900;
+  text-transform: uppercase;
+}
+
+#pulse-controls button[hidden] {
+  display: none;
+}
+
+#pulse-play-button {
+  background: linear-gradient(90deg, rgba(103, 244, 255, 0.9), rgba(210, 103, 255, 0.9));
+  color: #061120;
+}
+
 #poster-link:not([href]) {
   display: none;
diff --git a/tests/e2e/playable.spec.ts b/tests/e2e/playable.spec.ts
index 5cfd8c8..ed3061a 100644
--- a/tests/e2e/playable.spec.ts
+++ b/tests/e2e/playable.spec.ts
@@ -3,139 +3,153 @@ import { expect, test, type Page } from '@playwright/test';
 const WORLD_WIDTH = 1080;
 const WORLD_HEIGHT = 1920;
-const BLACK_HOLE_X = WORLD_WIDTH / 2;
-const BLACK_HOLE_Y = WORLD_HEIGHT * 0.44;

-test('help overlay opens on first visit and closes with play', async ({ page }) => {
+test('help opens on first visit', async ({ page }) => {
   await openGame(page);
   await expect(page.locator('#help-overlay')).toBeVisible();
-  await page.locator('#help-play-button').click();
-  await expect(page.locator('#help-overlay')).toBeHidden();
-  await page.waitForTimeout(250);
-  const snapshot = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot());
-  expect(snapshot?.timeMs).toBeGreaterThan(0);
+  await expect(page.locator('#help-title')).toHaveText('EVENT HORIZON');
+  await expect(page.locator('#help-overlay')).toContainText('Build a dark-energy chain');
```

```
 });

-test('smoke renders playable Pixi canvas', async ({ page }) => {
+test('connect two nodes by tap-tap', async ({ page }) => {
   await openGameAndPlay(page);
-  const canvas = page.locator('canvas');
-  await expect(canvas).toBeVisible();
-  await page.waitForTimeout(650);
-  const snapshot = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot());
-  expect(snapshot?.phase).toBeGreaterThanOrEqual(1);
-  expect(snapshot?.energy).toBeGreaterThan(0);
+  const nodes = await getNodes(page);
+  const source = nodes.find((node) => node.type === 'source');
+  const target = nodes.find((node) => node.type === 'energy');
+  expect(source).toBeTruthy();
+  expect(target).toBeTruthy();
+  await tapWorld(page, source!.x, source!.y);
+  await tapWorld(page, target!.x, target!.y);
+  const links = await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.getLinks()) as Array<{ fromId: number; toId: number }>;
+  expect(links.some((link) => link.fromId === source!.id && link.toId === target!.id)).toBe(true);
 });

-test('tap and tether swipe are captured into replay payload', async ({ page }) => {
-  await openGameAndPlay(page, '?debugInput=1');
-  const canvas = page.locator('canvas');
-  const box = await canvas.boundingBox();
-  expect(box).toBeTruthy();
-  if (!box) {
-    return;
-  }
-
-  await page.touchscreen.tap(box.x + box.width * 0.5, box.y + box.height * 0.5);
-  await waitForTutorialOrb(page);
-  const before = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot());
-  const swipe = await makeTutorialTetherSwipe(page);
-  await page.mouse.move(swipe.start.x, swipe.start.y);
+test('connect two nodes by drag', async ({ page }) => {
+  await openGameAndPlay(page);
+  const nodes = await getNodes(page);
+  const from = nodes.find((node) => node.type === 'energy');
+  const to = nodes.find((node) => node.type === 'delay');
+  expect(from).toBeTruthy();
+  expect(to).toBeTruthy();
+  const a = await worldToScreen(page, from!.x, from!.y);
+  const b = await worldToScreen(page, to!.x, to!.y);
+  await page.mouse.move(a.x, a.y);
   await page.mouse.down();
-  await page.mouse.move(swipe.mid.x, swipe.mid.y, { steps: 4 });
-  await page.mouse.move(swipe.end.x, swipe.end.y, { steps: 4 });
+  await page.mouse.move(b.x, b.y, { steps: 8 });
   await page.mouse.up();
-  await page.waitForTimeout(120);
-
-  const replay = await page.evaluate(() => window.__EVENT_HORIZON__?.getReplayPayload());
-  const after = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot());
-  const lastGesture = await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.getLastGesture());
-  const lastSwipe = replay?.swipeEvents.at(-1);
-  expect(replay?.tapEvents.length).toBeGreaterThanOrEqual(1);
-  expect(replay?.swipeEvents.length).toBeGreaterThanOrEqual(1);
-  expect(lastSwipe?.path.length).toBeGreaterThanOrEqual(2);
-  expect(lastSwipe?.target).not.toBe('empty');
-  expect(lastSwipe?.success).toBe(true);
-  expect(after?.score).toBeGreaterThan(before?.score ?? 0);
-  expect(lastGesture?.activeTrailCount).toBeGreaterThan(0);
+  const result = await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.getLastInputResult()) as { ok: boolean; message: string };
+  expect(result.ok).toBe(true);
 });

-test('posterizer exports a compact png data url', async ({ page }) => {
+test('press play, pulse moves, and energy node scores', async ({ page }) => {
   await openGameAndPlay(page);
-  await page.waitForTimeout(900);
-  const poster = await page.evaluate(() => window.__EVENT_HORIZON__?.exportPoster());
-  expect(poster).toMatch(/^data:image\/png;base64,/);
-  expect(poster?.length).toBeGreaterThan(1200);
+  await buildTutorialChain(page);
+  await page.locator('#pulse-play-button').click();
+  await page.waitForFunction(() => {
+    const snapshot = window.__EVENT_HORIZON__?.getSnapshot() as { phase?: string; pulses?: unknown[] };
+    return snapshot?.phase === 'pulse' && Number(snapshot.pulses?.length) > 0;
+  });
+  const before = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot()) as { score: number };
+  await page.waitForFunction((score) => {
+    const snapshot = window.__EVENT_HORIZON__?.getSnapshot() as { score?: number };
+    return Number(snapshot?.score) > Number(score);
+  }, before.score);
+  const after = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot()) as { score: number; multiplier: number; phase: string };
+  expect(after.phase).toBe('pulse');
+  expect(after.score).toBeGreaterThan(before.score);
+  expect(after.multiplier).toBeGreaterThanOrEqual(1);
 });

-test('miss swipe records visible feedback state', async ({ page }) => {
-  await openGameAndPlay(page, '?debugInput=1');
-  await page.waitForTimeout(250);
-  const result = await page.evaluate(() =>
-    window.__EVENT_HORIZON_DEBUG__?.simulateSwipeWorld([
-      { x: 80, y: 1760 },
-      { x: 180, y: 1840 }
-    ])
-  );
-  const lastGesture = await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.getLastGesture());
-  expect(result?.success).toBe(false);
-  expect(result?.message).toBe('MISS');
-  expect(lastGesture?.activeTrailCount).toBeGreaterThan(0);
+test('swipe during pulse phase creates Horizon Lens and records replay inputs', async ({ page }) => {
+  await openGameAndPlay(page);
+  await buildTutorialChain(page);
+  await page.locator('#pulse-play-button').click();
+  await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { phase?: string })?.phase === 'pulse');
+  const nodes = await getNodes(page);
+  const a = nodes.find((node) => node.id === 6) ?? nodes[4];
+  const b = nodes.find((node) => node.id === 8) ?? nodes[5];
+  const start = await worldToScreen(page, a.x, a.y);
+  const end = await worldToScreen(page, b.x, b.y);
+  await page.mouse.move(start.x, start.y);
+  await page.mouse.down();
+  await page.mouse.move((start.x + end.x) / 2, (start.y + end.y) / 2 - 30, { steps: 5 });
+  await page.mouse.move(end.x, end.y, { steps: 5 });
+  await page.mouse.up();
+  await page.waitForTimeout(120);
+  const result = await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.getLastInputResult()) as { kind: string; message: string };
+  const replay = await page.evaluate(() => window.__EVENT_HORIZON__?.getReplayPayload()) as {
+    buildInputs: unknown[];
+    liveInputs: Array<{ kind: string; success: boolean }>;
+  };
+  expect(result.kind).toBe('lens');
+  expect(['BRIDGE CREATED', 'NO ANCHOR']).toContain(result.message);
+  expect(replay.buildInputs.some((input) => (input as { kind: string }).kind === 'play')).toBe(true);
+  expect(replay.liveInputs.some((input) => input.kind === 'lens')).toBe(true);
 });

-test('end-state collapse is reachable', async ({ page }) => {
```

```
+test('collapse or stabilized end state is reachable', async ({ page }) => {
   await openGameAndPlay(page);
-  await page.evaluate(() => window.__EVENT_HORIZON__?.forceEnd());
-  await page.waitForTimeout(250);
-  const snapshot = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot());
-  expect(snapshot?.ended).toBe(true);
-  expect(snapshot?.collapseT).toBeGreaterThan(0);
+  await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.forceCollapse());
+  await page.waitForTimeout(150);
+  const snapshot = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot()) as {
+    phase: string;
+    collapsed: boolean;
+    stabilized: boolean;
+  };
+  expect(snapshot.phase).toBe('ended');
+  expect(snapshot.collapsed || snapshot.stabilized).toBe(true);
 });

-async function openGame(page: Page, query = ''): Promise<void> {
+async function openGame(page: Page): Promise<void> {
   await page.addInitScript(() => {
-    localStorage.clear();
+    window.localStorage.clear();
   });
-  await page.goto(query ? `.${query}` : './');
+  await page.goto('./?seed=tutorial&debugInput=1');
   await page.locator('canvas').waitFor({ state: 'visible' });
 }

-async function openGameAndPlay(page: Page, query = ''): Promise<void> {
-  await openGame(page, query);
-  await expect(page.locator('#help-overlay')).toBeVisible();
+async function openGameAndPlay(page: Page): Promise<void> {
+  await openGame(page);
   await page.locator('#help-play-button').click();
   await expect(page.locator('#help-overlay')).toBeHidden();
 }

-async function waitForTutorialOrb(page: Page) {
-  return page.waitForFunction(() => {
-    const snapshot = window.__EVENT_HORIZON__?.getSnapshot();
-    return snapshot?.orbs.some((orb) => orb.active && orb.tutorial && !orb.captured);
-  });
+async function buildTutorialChain(page: Page): Promise<void> {
+  const nodes = await getNodes(page);
+  const byId = new Map(nodes.map((node) => [node.id, node]));
+  for (const [fromId, toId] of [
+    [1, 2],
+    [2, 3],
+    [3, 4],
+    [4, 5],
+    [4, 6]
+  ]) {
+    const from = byId.get(fromId);
+    const to = byId.get(toId);
+    expect(from).toBeTruthy();
+    expect(to).toBeTruthy();
+    await tapWorld(page, from!.x, from!.y);
+    await tapWorld(page, to!.x, to!.y);
+  }
 }

-async function makeTutorialTetherSwipe(page: Page) {
-  const canvas = page.locator('canvas');
-  const box = await canvas.boundingBox();
+async function getNodes(page: Page) {
+  return (await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.getNodes())) as Array<{
+    id: number;
+    type: string;
+    x: number;
+    y: number;
+  }>;
+}
+
+async function tapWorld(page: Page, x: number, y: number): Promise<void> {
+  const point = await worldToScreen(page, x, y);
+  await page.mouse.click(point.x, point.y);
+}
+
+async function worldToScreen(page: Page, x: number, y: number): Promise<{ x: number; y: number }> {
+  const box = await page.locator('canvas').boundingBox();
   if (!box) {
     throw new Error('Canvas box unavailable.');
   }
-  const snapshot = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot());
-  const orb = snapshot?.orbs.find((candidate) => candidate.active && candidate.tutorial && !candidate.captured);
-  if (!orb) {
-    throw new Error('Tutorial orb unavailable.');
-  }
-  const targetX = BLACK_HOLE_X + (orb.x - BLACK_HOLE_X) * 0.78;
-  const targetY = BLACK_HOLE_Y + (orb.y - BLACK_HOLE_Y) * 0.78;
-  const dx = orb.x - BLACK_HOLE_X;
-  const dy = orb.y - BLACK_HOLE_Y;
-  const length = Math.max(1, Math.hypot(dx, dy));
-  const normalX = -dy / length;
-  const normalY = dx / length;
-  const start = worldToScreen(box, targetX - normalX * 150, targetY - normalY * 150);
-  const mid = worldToScreen(box, targetX, targetY);
-  const end = worldToScreen(box, targetX + normalX * 150, targetY + normalY * 150);
-  return { start, mid, end };
-}
-
-function worldToScreen(box: { x: number; y: number; width: number; height: number }, x: number, y: number) {
   const scale = Math.min(box.width / WORLD_WIDTH, box.height / WORLD_HEIGHT);
   const offsetX = (box.width - WORLD_WIDTH * scale) / 2;
diff --git a/tests/score-submit.test.ts b/tests/score-submit.test.ts
index 902edd5..c527df6 100644
--- a/tests/score-submit.test.ts
+++ b/tests/score-submit.test.ts
@@ -15,4 +15,39 @@ const replay = {
 };

+const pulseReplay = {
+  version: 1,
+  mode: 'pulse-chain',
+  seed: 'tutorial',
+  startedAt: 1780185600000,
+  buildInputs: [
+    { t: 0, kind: 'link', fromId: 1, toId: 2 },
+    { t: 220, kind: 'link', fromId: 2, toId: 3 },
+    { t: 520, kind: 'play' }
+  ],
+  liveInputs: [
+    {
+      t: 1420,
+      kind: 'lens',
+      path: [
+        { x: 835, y: 1215, t: 0 },
+        { x: 700, y: 1410, t: 110 }
+      ],
+      fromId: 6,
+      toId: 8,
+      success: true
```

```
+    }
+  ],
+  result: {
+    score: 620,
+    survivalMs: 12120,
+    maxMultiplier: 2,
+    loopsCompleted: 1,
+    linksUsed: 5,
+    stabilized: false,
+    collapsed: false
+  },
+  stepHash: '04ce9b1a'
+};
+
 describe('score-submit function', () => {
   it('accepts a valid replay payload', async () => {
@@ -28,4 +63,16 @@ describe('score-submit function', () => {
   });

+  it('accepts a valid pulse-chain replay payload', async () => {
+    const response = await scoreSubmit(
+      new Request('https://example.test/.netlify/functions/score-submit', {
+        method: 'POST',
+        headers: { 'content-type': 'application/json' },
+        body: JSON.stringify(pulseReplay)
+      })
+    );
+    expect(response.status).toBe(200);
+    await expect(response.json()).resolves.toMatchObject({ ok: true, score: 620, survivalMs: 12120 });
+  });
+
   it('rejects malformed replay payloads', async () => {
     const response = await scoreSubmit(
```

Full Source Code For Changed Text Files

README.md

```
# Event Horizon

Event Horizon is a mobile-first cosmic chain-reaction game about delaying a galaxy's collapse into a black hole. The current slice is a plain Vite + PixiJS v8 site with deterministic simulation state kept outside the renderer.

## Current Slice

- Logical portrait playfield: `1080 x 1920`
- PixiJS canvas renderer with WebGL preference
- Fixed `60 Hz` simulation step decoupled from render frames
- Seeded `mulberry32` RNG and replay payloads from seed + input timings
- Default Pulse Chain mode: connect Dark Energy Nodes, press Play, and watch a Stabilizing Pulse travel the network
- Tap-tap and drag link placement with a visible link budget, undo, clear, and reset
- Energy, Delay, Splitter, Conduit, and Source nodes with scoring and multiplier rules
- Horizon Lens swipes during pulse playback create short-lived temporary bridges
- Path-based input recording with mobile Pointer Events and TouchEvent fallback
- First-run help overlay, tutorial hints, visible pulse/lens feedback, and debug hooks
- Bottom Collapse Meter, score/multiplier HUD, and stabilized/collapsed end states
- Share poster export from three captured gameplay frames
- Minimal Netlify score submit function that accepts legacy and Pulse Chain replay payloads
- Previous tether survival prototype remains available with `?mode=legacy`

## Assumptions

- The initial repo was empty, so this is a greenfield Vite scaffold.
- Custom deterministic motion is enough for the Pulse Chain slice; no Matter.js or Planck.js dependency is needed.
- GitHub Pages is the public static target and uses `/event-horizon/` as the base path.
- Netlify deploys should set `VITE_BASE_PATH=/` through the included `netlify.toml`.
- The score function validates and acknowledges payloads only; durable storage is a backlog item.

## Commands

\`\`\`bash
npm install
npm run dev
npm run build
npm run lint
npm run test
npm run test:e2e
npm run score:test
npm run capture:iteration-03
npm run report:iteration-03
npm run capture:iteration-02
npm run report:iteration-02
npm run report:pdf
\`\`\`

Local Vite serves the game at:

\`\`\`text
http://127.0.0.1:5173/event-horizon/
\`\`\`

## Deployment

### GitHub Pages

The default Vite base path is `/event-horizon/`, so `npm run build` produces static assets compatible with:

\`\`\`text
https://kevinhegg.github.io/event-horizon/
\`\`\`

### Netlify

`netlify.toml` uses:

\`\`\`toml
[build]
  command = "VITE_BASE_PATH=/ npm run build"
  publish = "dist"

[functions]
  directory = "netlify/functions"
\`\`\`

The score endpoint is available at:

\`\`\`text
/.netlify/functions/score-submit
\`\`\`

For local Netlify function routing, run Netlify CLI from the repo root:

\`\`\`bash
npx netlify dev
\`\`\`

## Replay Payload Shape

\`\`\`json
```

```
{
  "version": 1,
  "mode": "pulse-chain",
  "seed": "tutorial",
  "startedAt": 1780185600000,
  "buildInputs": [
    { "t": 0, "kind": "link", "fromId": 1, "toId": 2 },
    { "t": 300, "kind": "link", "fromId": 2, "toId": 3 },
    { "t": 620, "kind": "play" }
  ],
  "liveInputs": [
    {
      "t": 1600,
      "kind": "lens",
      "path": [
        { "x": 835, "y": 1215, "t": 0 },
        { "x": 700, "y": 1410, "t": 120 }
      ],
      "fromId": 6,
      "toId": 8,
      "success": true
    }
  ],
  "result": {
    "score": 620,
    "survivalMs": 12120,
    "maxMultiplier": 2,
    "loopsCompleted": 1,
    "linksUsed": 5,
    "stabilized": false,
    "collapsed": false
  },
  "stepHash": "04ce9b1a"
}
\`\`\`

Legacy mode still uses:

\`\`\`json
{
  "version": 1,
  "seed": "eh-2026-05-29-alpha",
  "startedAt": 1780051200000,
  "survivalMs": 67421,
  "score": 1280,
  "energyCaptured": 87,
  "maxStreak": 24,
  "tapEvents": [],
  "swipeEvents": [],
  "phaseTransitions": []
}
\`\`\`

## Current Branch Workflow

\`\`\`bash
git switch -c feat/pulse-chain-pivot
git add .
git commit -m "Pivot Event Horizon to pulse chain gameplay"
git push -u origin feat/pulse-chain-pivot
gh pr create --base main --head feat/pulse-chain-pivot --title "Pivot Event Horizon to pulse chain gameplay" --body-file docs/iteration-03-report.md
\`\`\`
```

index.html

```
<!doctype html>
<html lang="en-US">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover" />
    <meta name="theme-color" content="#03040a" />
    <meta name="description" content="Event Horizon, a one-thumb galaxy survival prototype." />
    <title>Event Horizon</title>
  </head>
  <body>
    <main id="app" aria-label="Event Horizon game">
      <div id="game-shell">
        <div id="game-root" aria-label="Playable canvas"></div>
        <div id="status-layer" aria-live="polite">
          <button id="restart-button" type="button" aria-label="Restart run" title="Restart run">↻</button>
          <button id="share-button" type="button" aria-label="Create share poster" title="Create share poster">⇪</button>
          <button id="help-button" type="button" aria-label="How to play" title="How to play">?</button>
          <a id="poster-link" download="event-horizon-poster.png" aria-label="Download share poster">⇩</a>
        </div>
        <div id="pulse-controls" aria-live="polite">
          <button id="pulse-undo-button" type="button">Undo</button>
          <button id="pulse-clear-button" type="button">Clear</button>
          <button id="pulse-play-button" type="button">Play</button>
        </div>
        <section id="help-overlay" aria-modal="true" role="dialog" aria-labelledby="help-title" hidden>
          <div id="help-panel">
            <div class="help-example" aria-hidden="true">
              <span class="help-hole"></span>
              <span class="help-tether"></span>
              <span class="help-orb"></span>
              <span class="help-swipe"></span>
            </div>
            <h1 id="help-title">EVENT HORIZON</h1>
            <p>Build a dark-energy chain. Then press Play.</p>
            <ol>
              <li>Connect nodes with gravitational links.</li>
              <li>Press Play to launch the stabilizing pulse.</li>
              <li>Energy nodes refill the Collapse Meter.</li>
              <li>Long chains and loops build multipliers.</li>
              <li>During playback, swipe to create a temporary Horizon Lens bridge.</li>
              <li>Keep the galaxy alive as long as you can.</li>
            </ol>
            <p>The black hole always wins.<br />Your chain buys the galaxy time.</p>
            <button id="help-play-button" type="button">PLAY</button>
          </div>
        </section>
      </div>
    </main>
    <script type="module" src="/src/main.ts"></script>
  </body>
</html>
```

netlify/functions/score-submit.mjs

```
const corsHeaders = {
  'access-control-allow-origin': '*',
  'access-control-allow-methods': 'POST, OPTIONS',
  'access-control-allow-headers': 'content-type',
  'content-type': 'application/json'
};

export default async function scoreSubmit(request) {
```

```
  if (request.method === 'OPTIONS') {
    return json({ ok: true }, 204);
  }

  if (request.method !== 'POST') {
    return json({ ok: false, error: 'method_not_allowed' }, 405);
  }

  let payload;
  try {
    payload = await request.json();
  } catch {
    return json({ ok: false, error: 'invalid_json' }, 400);
  }

  const validationError = validateReplay(payload);
  if (validationError) {
    return json({ ok: false, error: validationError }, 422);
  }

  return json({
    ok: true,
    acceptedAt: new Date().toISOString(),
    score: payload.mode === 'pulse-chain' ? payload.result.score : payload.score,
    survivalMs: payload.mode === 'pulse-chain' ? payload.result.survivalMs : payload.survivalMs
  });
}

function json(body, status = 200) {
  return new Response(JSON.stringify(body), {
    status,
    headers: corsHeaders
  });
}

function validateReplay(payload) {
  if (!payload || typeof payload !== 'object') {
    return 'payload_required';
  }
  if (payload.version !== 1) {
    return 'version_required';
  }
  if (payload.mode === 'pulse-chain') {
    if (typeof payload.seed !== 'string' || payload.seed.length < 3) {
      return 'seed_required';
    }
    if (!Number.isFinite(payload.startedAt)) {
      return 'started_at_required';
    }
    if (!Array.isArray(payload.buildInputs) || !Array.isArray(payload.liveInputs)) {
      return 'inputs_required';
    }
    if (!payload.result || !Number.isFinite(payload.result.score) || !Number.isFinite(payload.result.survivalMs)) {
      return 'result_required';
    }
    if (typeof payload.stepHash !== 'string') {
      return 'hash_required';
    }
    return '';
  }
  if (typeof payload.seed !== 'string' || payload.seed.length < 3) {
    return 'seed_required';
  }
  if (!Number.isFinite(payload.startedAt)) {
    return 'started_at_required';
  }
  if (!Number.isFinite(payload.survivalMs) || payload.survivalMs < 0) {
    return 'survival_required';
  }
  if (!Number.isFinite(payload.score) || payload.score < 0) {
    return 'score_required';
  }
  if (!Array.isArray(payload.tapEvents) || !Array.isArray(payload.swipeEvents)) {
    return 'inputs_required';
  }
  return '';
}
```

package.json

```
{
  "name": "event-horizon",
  "version": "0.1.0",
  "private": true,
  "type": "module",
  "scripts": {
    "dev": "vite --host 127.0.0.1",
    "build": "tsc --noEmit && vite build",
    "preview": "vite preview --host 127.0.0.1",
    "test": "vitest run",
    "test:e2e": "playwright test",
    "lint": "eslint .",
    "score:test": "node scripts/test-score-submit.mjs",
    "capture:artifacts": "node scripts/capture-artifacts.mjs",
    "capture:iteration-02": "node scripts/capture-iteration-02-artifacts.mjs",
    "capture:iteration-03": "node scripts/capture-iteration-03-artifacts.mjs",
    "report:pdf": "node scripts/generate-report-pdf.mjs",
    "report:iteration-02": "node scripts/generate-iteration-02-report.mjs",
    "report:iteration-03": "node scripts/generate-iteration-03-report.mjs"
  },
  "dependencies": {
    "pixi.js": "8.18.1"
  },
  "devDependencies": {
    "@eslint/js": "9.39.1",
    "@playwright/test": "1.57.0",
    "@types/node": "24.12.4",
    "eslint": "9.39.1",
    "typescript": "5.9.3",
    "typescript-eslint": "8.49.0",
    "vite": "8.0.14",
    "vitest": "4.0.15"
  }
}
```

scripts/capture-iteration-03-artifacts.mjs

```
import { mkdir } from 'node:fs/promises';
import { chromium } from '@playwright/test';

const WORLD_WIDTH = 1080;
const WORLD_HEIGHT = 1920;
const outputDir = new URL('../docs/artifacts/', import.meta.url);

await mkdir(outputDir, { recursive: true });

const browser = await chromium.launch({ channel: 'chrome' });
```

```
const page = await browser.newPage({
  viewport: { width: 390, height: 844 },
  deviceScaleFactor: 2,
  isMobile: true,
  hasTouch: true
});

await page.addInitScript(() => window.localStorage.clear());
await page.goto('http://127.0.0.1:5173/event-horizon/?seed=tutorial&debugInput=1', { waitUntil: 'networkidle' });
await page.locator('canvas').waitFor({ state: 'visible', timeout: 10000 });
await screenshot('iteration-03-help-mobile.jpg', 76);

await page.locator('#help-play-button').click();
await page.waitForFunction(() => window.__EVENT_HORIZON__?.getSnapshot()?.phase === 'build');
await page.waitForTimeout(220);
await screenshot('iteration-03-build-phase-mobile.jpg', 78);

await buildTutorialChain();
await page.waitForTimeout(180);
await screenshot('iteration-03-link-placement-mobile.jpg', 78);

await page.locator('#pulse-play-button').click();
await page.waitForFunction(() => {
  const snapshot = window.__EVENT_HORIZON__?.getSnapshot();
  return snapshot?.phase === 'pulse' && snapshot.pulses.length > 0;
});
await page.waitForTimeout(850);
await screenshot('iteration-03-pulse-running-mobile.jpg', 78);

await makeLensSwipe(6, 8);
await page.waitForTimeout(140);
await screenshot('iteration-03-horizon-lens-mobile.jpg', 80);

await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.forceCollapse());
await page.waitForTimeout(260);
await screenshot('iteration-03-end-screen-mobile.jpg', 72);

await browser.close();

for (const file of [
  'iteration-03-help-mobile.jpg',
  'iteration-03-build-phase-mobile.jpg',
  'iteration-03-link-placement-mobile.jpg',
  'iteration-03-pulse-running-mobile.jpg',
  'iteration-03-horizon-lens-mobile.jpg',
  'iteration-03-end-screen-mobile.jpg'
]) {
  console.log(`Captured docs/artifacts/${file}`);
}

async function buildTutorialChain() {
  for (const [fromId, toId] of [
    [1, 2],
    [2, 3],
    [3, 4],
    [4, 5],
    [4, 6]
  ]) {
    const from = await nodeById(fromId);
    const to = await nodeById(toId);
    await tapWorld(from.x, from.y);
    await tapWorld(to.x, to.y);
    await page.waitForTimeout(50);
  }
}

async function makeLensSwipe(fromId, toId) {
  const from = await nodeById(fromId);
  const to = await nodeById(toId);
  const start = await worldToScreen(from.x, from.y);
  const end = await worldToScreen(to.x, to.y);
  await page.mouse.move(start.x, start.y);
  await page.mouse.down();
  await page.mouse.move((start.x + end.x) / 2, (start.y + end.y) / 2 - 42, { steps: 6 });
  await page.mouse.move(end.x, end.y, { steps: 6 });
  await page.mouse.up();
}

async function nodeById(id) {
  const node = await page.evaluate((nodeId) => window.__EVENT_HORIZON_DEBUG__?.getNodes().find((candidate) => candidate.id === nodeId), id);
  if (!node) {
    throw new Error(`Node ${id} not found.`);
  }
  return node;
}

async function tapWorld(x, y) {
  const point = await worldToScreen(x, y);
  await page.mouse.click(point.x, point.y);
}

async function worldToScreen(x, y) {
  const box = await page.locator('canvas').boundingBox();
  if (!box) {
    throw new Error('Canvas box unavailable.');
  }
  const scale = Math.min(box.width / WORLD_WIDTH, box.height / WORLD_HEIGHT);
  const offsetX = (box.width - WORLD_WIDTH * scale) / 2;
  const offsetY = (box.height - WORLD_HEIGHT * scale) / 2;
  return {
    x: box.x + offsetX + x * scale,
    y: box.y + offsetY + y * scale
  };
}

async function screenshot(file, quality) {
  await page.screenshot({
    path: new URL(file, outputDir).pathname,
    type: 'jpeg',
    quality,
    fullPage: false
  });
}
```

scripts/generate-iteration-03-report.mjs

```
import { execFileSync } from 'node:child_process';
import { existsSync } from 'node:fs';
import { mkdir, readFile, stat, writeFile } from 'node:fs/promises';
import { extname } from 'node:path';
import { chromium } from '@playwright/test';

const repoRoot = new URL('../', import.meta.url);
const docsDir = new URL('../docs/', import.meta.url);
const artifactsDir = new URL('artifacts/', docsDir);
const reportPath = new URL('iteration-03-report.md', docsDir);
const pdfPath = new URL('iteration-03-report.pdf', docsDir);
const testResultsPath = new URL('artifacts/iteration-03-test-results.txt', docsDir);
const baseRef = process.env.REPORT_BASE_REF ?? 'origin/main';
const screenshotFiles = [
```

```
  ['Help overlay', 'iteration-03-help-mobile.jpg'],
  ['Build phase', 'iteration-03-build-phase-mobile.jpg'],
  ['Link placement', 'iteration-03-link-placement-mobile.jpg'],
  ['Pulse running', 'iteration-03-pulse-running-mobile.jpg'],
  ['Horizon Lens bridge', 'iteration-03-horizon-lens-mobile.jpg'],
  ['End screen', 'iteration-03-end-screen-mobile.jpg']
];

await mkdir(docsDir, { recursive: true });

const changedFiles = await collectChangedFiles();
const sourceFiles = changedFiles.filter((file) => isTextSource(file));
const binaryFiles = changedFiles.filter((file) => !isTextSource(file));
const testResults = existsSync(testResultsPath)
  ? await readFile(testResultsPath, 'utf8')
  : 'Test result log was not present when this report was generated.';
const diffStat = git(['diff', '--stat', `${baseRef}...HEAD`]) || git(['diff', '--stat']);
const currentDiff = git(['diff', '--no-ext-diff', '--unified=2']);
const stagedDiff = git(['diff', '--cached', '--no-ext-diff', '--unified=2']);

const markdown = await buildMarkdown({
  sourceFiles,
  binaryFiles,
  testResults,
  diffStat,
  diffs: [stagedDiff, currentDiff].filter(Boolean).join('\n')
});
await writeFile(reportPath, markdown);

const html = await buildHtml(markdown, sourceFiles);
const browser = await chromium.launch({ channel: 'chrome' });
const page = await browser.newPage();
await page.setContent(html, { waitUntil: 'load' });
await page.pdf({
  path: pdfPath.pathname,
  format: 'Letter',
  printBackground: true,
  preferCSSPageSize: true,
  margin: {
    top: '0.42in',
    right: '0.38in',
    bottom: '0.42in',
    left: '0.38in'
  }
});
await browser.close();

const size = await stat(pdfPath);
console.log('Wrote docs/iteration-03-report.md');
console.log(`Wrote docs/iteration-03-report.pdf (${Math.round(size.size / 1024)} KiB)`);

async function collectChangedFiles() {
  const names = new Set([
    ...lines(git(['diff', `${baseRef}...HEAD`, '--name-only'])),
    ...lines(git(['diff', '--cached', '--name-only'])),
    ...lines(git(['diff', '--name-only'])),
    ...lines(git(['ls-files', '--others', '--exclude-standard']))
  ]);
  names.delete('docs/iteration-03-report.md');
  names.delete('docs/iteration-03-report.pdf');
  return [...names].sort();
}

async function buildMarkdown({ sourceFiles, binaryFiles, testResults, diffStat, diffs }) {
  const sourceBlocks = [];
  for (const file of sourceFiles) {
    const content = await readFile(new URL(file, repoRoot), 'utf8');
    sourceBlocks.push(`### ${file}\n\n\`\`\`${languageFor(file)}\n${content.replaceAll('``\`', '``\\`\\`')}\n\`\`\``);
  }

  return `# Event Horizon Iteration 03 Report

## Summary Of Pivot

Iteration 03 pivots Event Horizon from tether-swiping survival into the default Pulse Chain mode: connect Dark Energy Nodes with Gravitational Links, press Play, watch a Stabilizing Pulse
traverse the network, and rescue the run with short-lived Horizon Lens bridges during playback.

## Why The Old Loop Was Not Fun Enough

The earlier prototype improved mobile input, but its main action still felt like touching a moving target. It asked for precision during chaos before the player understood the plan. Pulse Chain
moves the fun toward planning, payoff, readable cause and effect, and one live skill action that supports the puzzle instead of becoming the entire challenge.

## New Gameplay Explanation

- Build phase: tap-tap or drag from one node to another to place a directional Gravitational Link.
- Pulse phase: press Play to launch a Stabilizing Pulse from the Source Node.
- Scoring: Energy Nodes refill the Collapse Meter, long chains raise multipliers, Splitter Nodes branch pulses, and Delay Nodes hold timing.
- Rescue phase: swipe during playback to draw a Horizon Lens. If the swipe anchors near two valid nodes, it creates a temporary bridge.
- End state: reaching the score or survival target stabilizes the sector; empty energy collapses the galaxy.

## Exact Files Changed

${[...sourceFiles, ...binaryFiles].map((file) => `- ${file}`).join('\n')}

## Diff Summary

\`\`\`text
${diffStat || 'No committed diff stat was available at report generation time.').trim()}
\`\`\`

## Tests Run And Results

\`\`\`text
${testResults.trim()}
\`\`\`

## Screenshots

${screenshotFiles.map(([, file]) => `- docs/artifacts/${file}`).join('\n')}

## Known Limitations

- Playwright mobile simulation passed; this run did not include hands-on testing on a physical phone.
- Horizon Lens uses the understandable temporary-link mechanic for this iteration, not full pulse deflection.
- When all pulses die, the run currently ends with accelerated collapse pressure represented as a pulse-died end reason; future tuning should return players to build/retry more gracefully.
- Node labels are visualized mostly through shape and color; richer in-canvas labels and sound are good next steps.
- GitHub Pages serves the static game only. Netlify Functions remain available only on Netlify or local Netlify dev.

## Next Recommended Iteration

- Tune the tutorial seed on a real iPhone and Android phone.
- Add sound, haptics, and stronger pulse arrival bursts.
- Add a retry flow that preserves the same seed and highlights the failed dead end.
- Build a replay viewer for the Pulse Chain payload.
- Add leaderboards or local seed challenge sharing once the loop feels sticky.

## Full Diffs For Tracked Changes

\`\`\`diff
${diffs.trim() || 'No tracked working-tree diff was present. Untracked files are included in full-source sections below.'}
\`\`\`

## Full Source Code For Changed Text Files
```

```
${sourceBlocks.join('\n\n')}
`;
}

async function buildHtml(markdown, sourceFiles) {
  const images = await Promise.all(
    screenshotFiles.map(async ([label, file]) => {
      const url = new URL(file, artifactsDir);
      if (!existsSync(url)) {
        return '';
      }
      const bytes = await readFile(url);
      return `<figure><img src="${bytes.toString('base64')}" alt="${escapeHtml(label)}" /><figcaption>${escapeHtml(label)}</figcaption></figure>`;
    })
  );

  return `<!doctype html>
<html lang="en-US">
<head>
  <meta charset="utf-8" />
  <title>Event Horizon Iteration 03 Report</title>
  <style>
    @page { size: Letter; margin: 0.42in 0.38in; }
    * { box-sizing: border-box; }
    body {
      margin: 0;
      color: #15202b;
      font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
      font-size: 10px;
      line-height: 1.34;
    }
    h1, h2, h3 {
      color: #07111f;
      line-height: 1.12;
      margin: 0.58rem 0 0.28rem;
      break-after: avoid;
      page-break-after: avoid;
    }
    h1 { font-size: 21px; border-bottom: 2px solid #2f7ea1; padding-bottom: 7px; }
    h2 { font-size: 14.5px; }
    h3 { font-size: 11px; }
    p { margin: 0 0 0.4rem; }
    ul, ol { margin: 0 0 0.5rem 1.08rem; padding: 0; }
    li { margin: 0.08rem 0; }
    code { font-family: "SFMono-Regular", Menlo, Consolas, monospace; font-size: 7.8px; }
    pre {
      margin: 0.32rem 0 0.62rem;
      padding: 7px;
      white-space: pre-wrap;
      overflow-wrap: anywhere;
      word-break: break-word;
      background: #f3f6f9;
      border: 1px solid #d9e2ea;
      border-radius: 5px;
      font-family: "SFMono-Regular", Menlo, Consolas, monospace;
      font-size: 6.2px;
      line-height: 1.18;
      break-inside: auto;
      page-break-inside: auto;
    }
    a { color: #0b6f9c; text-decoration: none; }
    .cover {
      background: #07111f;
      color: #eef9ff;
      border-radius: 8px;
      padding: 17px;
      margin-bottom: 11px;
      break-inside: avoid;
    }
    .cover h1 { color: #fff; border-color: #80e3ff; margin-top: 0; }
    .cover p { color: #cbeaf4; margin-bottom: 0; }
    .gallery {
      display: grid;
      grid-template-columns: repeat(3, 1fr);
      gap: 8px;
      margin: 8px 0 12px;
      break-inside: avoid;
    }
    figure { margin: 0; break-inside: avoid; }
    img {
      display: block;
      width: 100%;
      max-height: 235px;
      object-fit: contain;
      border: 1px solid #d9e2ea;
      border-radius: 6px;
      background: #03040a;
    }
    figcaption { margin-top: 3px; color: #526171; font-size: 8.4px; text-align: center; }
    .source-index { columns: 2; column-gap: 22px; }
  </style>
</head>
<body>
  <section class="cover">
    <h1>Event Horizon Iteration 03 Report</h1>
    <p>Pulse Chain pivot: planning, payoff, and Horizon Lens rescue play.</p>
  </section>
  <section>
    <h2>Screenshots</h2>
    <div class="gallery">${images.join('')}</div>
  </section>
  <section>
    <h2>Changed Source Index</h2>
    <ul class="source-index">${sourceFiles.map((file) => `<li>${escapeHtml(file)}</li>`).join('')}</ul>
  </section>
  ${markdownToHtml(markdown)}
</body>
</html>`;
}

function markdownToHtml(markdown) {
  const rows = markdown.split('\n');
  const html = [];
  let inCode = false;
  let codeLines = [];
  let listOpen = false;

  for (const row of rows) {
    if (row.startsWith('```')) {
      if (inCode) {
        html.push(`<pre>${escapeHtml(codeLines.join('\n'))}</pre>`);
        codeLines = [];
      }
      inCode = !inCode;
      continue;
    }

    if (inCode) {
      codeLines.push(row);
      continue;
    }
```

```
    if (row.startsWith('- ')) {
      if (!listOpen) {
        html.push('<ul>');
        listOpen = true;
      }
      html.push(`<li>${inlineMarkdown(row.slice(2))}</li>`);
      continue;
    }

    if (listOpen) {
      html.push('</ul>');
      listOpen = false;
    }

    if (row.startsWith('# ')) {
      html.push(`<h1>${inlineMarkdown(row.slice(2))}</h1>`);
    } else if (row.startsWith('## ')) {
      html.push(`<h2>${inlineMarkdown(row.slice(3))}</h2>`);
    } else if (row.startsWith('### ')) {
      html.push(`<h3>${inlineMarkdown(row.slice(4))}</h3>`);
    } else if (/^\d+\.\s/.test(row)) {
      html.push(`<p>${inlineMarkdown(row)}</p>`);
    } else if (row.trim()) {
      html.push(`<p>${inlineMarkdown(row)}</p>`);
    }
  }

  if (listOpen) {
    html.push('</ul>');
  }
  return html.join('\n');
}

function inlineMarkdown(value) {
  return escapeHtml(value)
    .replace(/`([^`]+)`/g, '<code>$1</code>')
    .replace(/\[([^\]]+)\]\(([^)]+)\)/g, '<a href="$2">$1</a>');
}

function escapeHtml(value) {
  return String(value)
    .replaceAll('&', '&amp;')
    .replaceAll('<', '&lt;')
    .replaceAll('>', '&gt;')
    .replaceAll('"', '&quot;')
    .replaceAll("'", '&#39;');
}

function git(args) {
  try {
    return execFileSync('git', args, {
      cwd: repoRoot,
      encoding: 'utf8',
      maxBuffer: 1024 * 1024 * 32
    });
  } catch {
    return '';
  }
}

function lines(value) {
  return value
    .split('\n')
    .map((line) => line.trim())
    .filter(Boolean);
}

function isTextSource(file) {
  if (file.startsWith('docs/artifacts/') || file.endsWith('.pdf')) {
    return false;
  }
  return ['.css', '.html', '.js', '.json', '.md', '.mjs', '.ts', '.tsx', '.txt', '.yml'].includes(extname(file));
}

function languageFor(file) {
  const extension = extname(file);
  if (extension === '.ts' || extension === '.tsx') {
    return 'ts';
  }
  if (extension === '.mjs' || extension === '.js') {
    return 'js';
  }
  if (extension === '.css') {
    return 'css';
  }
  if (extension === '.html') {
    return 'html';
  }
  if (extension === '.json') {
    return 'json';
  }
  if (extension === '.yml') {
    return 'yaml';
  }
  if (extension === '.md') {
    return 'md';
  }
  return 'text';
}
```

scripts/test-score-submit.mjs

```
import scoreSubmit from '../netlify/functions/score-submit.mjs';

const replay = {
  version: 1,
  seed: 'eh-2026-05-29-alpha',
  startedAt: 1780051200000,
  survivalMs: 67421,
  score: 1280,
  energyCaptured: 87,
  maxStreak: 24,
  tapEvents: [],
  swipeEvents: [],
  phaseTransitions: []
};

const pulseReplay = {
  version: 1,
  mode: 'pulse-chain',
  seed: 'tutorial',
  startedAt: 1780185600000,
  buildInputs: [
    { t: 0, kind: 'link', fromId: 1, toId: 2 },
    { t: 300, kind: 'link', fromId: 2, toId: 3 },
    { t: 620, kind: 'play' }
  ],
  liveInputs: [
    {
      t: 1600,
```

```
      kind: 'lens',
      path: [
        { x: 835, y: 1215, t: 0 },
        { x: 700, y: 1410, t: 120 }
      ],
      fromId: 6,
      toId: 8,
      success: true
    }
  ],
  result: {
    score: 620,
    survivalMs: 12120,
    maxMultiplier: 2,
    loopsCompleted: 1,
    linksUsed: 5,
    stabilized: false,
    collapsed: false
  },
  stepHash: '04ce9b1a'
};

const results = [];
for (const payload of [replay, pulseReplay]) {
  const response = await scoreSubmit(
    new Request('https://event-horizon.test/.netlify/functions/score-submit', {
      method: 'POST',
      headers: { 'content-type': 'application/json' },
      body: JSON.stringify(payload)
    })
  );
  const body = await response.json();
  const expectedScore = payload.mode === 'pulse-chain' ? payload.result.score : payload.score;
  if (response.status !== 200 || !body.ok || body.score !== expectedScore) {
    console.error(JSON.stringify({ status: response.status, body }, null, 2));
    process.exit(1);
  }
  results.push({ status: response.status, body });
}

console.log(JSON.stringify(results, null, 2));
```

src/game/pulse/PulseGeometry.ts

```
import { clamp, distanceSquared, distanceToSegmentSquared } from '../math';
import type { PulseNode } from './PulseTypes';
import type { WorldPoint } from '../gestures';

export function simplifyPath<T extends WorldPoint>(points: readonly T[], maxPoints = 24): T[] {
  if (points.length <= maxPoints) {
    return points.map((point) => point);
  }
  const sampled: T[] = [];
  for (let index = 0; index < maxPoints; index += 1) {
    sampled.push(points[Math.round((index / (maxPoints - 1)) * (points.length - 1))]);
  }
  return sampled;
}

export function resamplePath(points: readonly WorldPoint[], spacing = 38): WorldPoint[] {
  if (points.length <= 1) {
    return points.map((point) => ({ ...point }));
  }
  const result: WorldPoint[] = [{ ...points[0] }];
  let carry = 0;
  for (let index = 1; index < points.length; index += 1) {
    const previous = points[index - 1];
    const point = points[index];
    const segmentLength = Math.max(0.001, Math.hypot(point.x - previous.x, point.y - previous.y));
    let distance = spacing - carry;
    while (distance <= segmentLength) {
      const t = distance / segmentLength;
      result.push({
        x: previous.x + (point.x - previous.x) * t,
        y: previous.y + (point.y - previous.y) * t
      });
      distance += spacing;
    }
    carry = segmentLength - (distance - spacing);
  }
  const last = points[points.length - 1];
  result.push({ ...last });
  return result;
}

export function smoothPathQuadratic(points: readonly WorldPoint[]): string {
  if (points.length === 0) {
    return '';
  }
  if (points.length === 1) {
    return `M ${points[0].x} ${points[0].y}`;
  }
  const commands = [`M ${points[0].x} ${points[0].y}`];
  for (let index = 1; index < points.length - 1; index += 1) {
    const point = points[index];
    const next = points[index + 1];
    const midpoint = midpointOf(point, next);
    commands.push(`Q ${point.x} ${point.y} ${midpoint.x} ${midpoint.y}`);
  }
  const last = points[points.length - 1];
  commands.push(`L ${last.x} ${last.y}`);
  return commands.join(' ');
}

export function distancePointToSegment(point: WorldPoint, start: WorldPoint, end: WorldPoint): number {
  return Math.sqrt(distanceToSegmentSquared(point.x, point.y, start.x, start.y, end.x, end.y));
}

export function distanceSegmentToSegment(a: WorldPoint, b: WorldPoint, c: WorldPoint, d: WorldPoint): number {
  if (segmentsIntersect(a, b, c, d)) {
    return 0;
  }
  return Math.sqrt(
    Math.min(
      distanceToSegmentSquared(a.x, a.y, c.x, c.y, d.x, d.y),
      distanceToSegmentSquared(b.x, b.y, c.x, c.y, d.x, d.y),
      distanceToSegmentSquared(c.x, c.y, a.x, a.y, b.x, b.y),
      distanceToSegmentSquared(d.x, d.y, a.x, a.y, b.x, b.y)
    )
  );
}

export function nearestNodeToPath(
  nodes: readonly PulseNode[],
  path: readonly WorldPoint[],
  radius = 118,
  excludedId?: number
): PulseNode | undefined {
  let best: PulseNode | undefined;
  let bestDistance = radius;
```

```
  for (const node of nodes) {
    if (node.id === excludedId) {
      continue;
    }
    const distance = distanceNodeToPath(node, path);
    if (distance <= bestDistance) {
      best = node;
      bestDistance = distance;
    }
  }
  return best;
}

export function nearestTwoNodesToPath(
  nodes: readonly PulseNode[],
  path: readonly WorldPoint[],
  radius = 126
): [PulseNode, PulseNode] | undefined {
  const ranked = nodes
    .map((node) => ({ node, distance: distanceNodeToPath(node, path) }))
    .filter((entry) => entry.distance <= radius)
    .sort((a, b) => a.distance - b.distance);
  if (ranked.length < 2) {
    return undefined;
  }
  return [ranked[0].node, ranked[1].node];
}

export function pathCrossesNodeRadius(path: readonly WorldPoint[], node: PulseNode, radius = node.radius + 44): boolean {
  return distanceNodeToPath(node, path) <= radius;
}

export function distanceNodeToPath(node: PulseNode, path: readonly WorldPoint[]): number {
  if (path.length === 0) {
    return Number.POSITIVE_INFINITY;
  }
  if (path.length === 1) {
    return Math.sqrt(distanceSquared(node.x, node.y, path[0].x, path[0].y));
  }
  let best = Number.POSITIVE_INFINITY;
  for (let index = 1; index < path.length; index += 1) {
    const previous = path[index - 1];
    const point = path[index];
    best = Math.min(best, distancePointToSegment(node, previous, point));
  }
  return best;
}

export function curvedLinkPath(from: WorldPoint, to: WorldPoint, bend = 0.12): WorldPoint[] {
  const mid = midpointOf(from, to);
  const dx = to.x - from.x;
  const dy = to.y - from.y;
  const length = Math.max(1, Math.hypot(dx, dy));
  const blackHolePull = 0.18;
  const control = {
    x: mid.x - (dy / length) * length * bend + (540 - mid.x) * blackHolePull,
    y: mid.y + (dx / length) * length * bend + (845 - mid.y) * blackHolePull
  };
  return [from, control, to];
}

export function pointOnQuadratic(a: WorldPoint, b: WorldPoint, c: WorldPoint, t: number): WorldPoint {
  const clamped = clamp(t, 0, 1);
  const inv = 1 - clamped;
  return {
    x: inv * inv * a.x + 2 * inv * clamped * b.x + clamped * clamped * c.x,
    y: inv * inv * a.y + 2 * inv * clamped * b.y + clamped * clamped * c.y
  };
}

function midpointOf(a: WorldPoint, b: WorldPoint): WorldPoint {
  return {
    x: (a.x + b.x) / 2,
    y: (a.y + b.y) / 2
  };
}

function segmentsIntersect(a: WorldPoint, b: WorldPoint, c: WorldPoint, d: WorldPoint): boolean {
  const o1 = orientation(a, b, c);
  const o2 = orientation(a, b, d);
  const o3 = orientation(c, d, a);
  const o4 = orientation(c, d, b);
  if (o1 !== o2 && o3 !== o4) {
    return true;
  }
  return false;
}

function orientation(a: WorldPoint, b: WorldPoint, c: WorldPoint): -1 | 0 | 1 {
  const value = (b.y - a.y) * (c.x - b.x) - (b.x - a.x) * (c.y - b.y);
  if (Math.abs(value) < 0.000001) {
    return 0;
  }
  return value > 0 ? 1 : -1;
}
```

src/game/pulse/PulseInputController.ts

```
import { distanceSquared } from '../math';
import { InputHandler, type GesturePoint, type SwipeGesture, type WorldPoint } from '../InputHandler';
import type { PulseSimulation } from './PulseSimulation';
import type { PulseInputResult, PulseNode } from './PulseTypes';

export interface PulseInputViewState {
  selectedNodeId?: number;
  nearestNodeId?: number;
  previewFromId?: number;
  previewPoint?: WorldPoint;
  liveGesture: readonly WorldPoint[];
  lastResult: PulseInputResult;
}

export class PulseInputController {
  private input?: InputHandler;
  private selectedNodeId: number | undefined;
  private previewFromId: number | undefined;
  private previewPoint: WorldPoint | undefined;
  private nearestNodeId: number | undefined;
  private liveGesture: WorldPoint[] = [];
  private lastResult: PulseInputResult = { ok: true, kind: 'none', message: 'CONNECT TWO NODES' };
  private debug = false;

  constructor(
    private readonly target: HTMLElement,
    private readonly sim: PulseSimulation,
    private readonly screenToWorld: (clientX: number, clientY: number) => WorldPoint
  ) {}

  start(): void {
    this.input = new InputHandler(this.target, {
```

```
      screenToWorld: this.screenToWorld,
      onTap: (point) => this.handleTap(point),
      onSwipe: (gesture) => this.handleSwipe(gesture),
      onGesturePreview: (points) => this.handlePreview(points),
      onGestureEnd: () => this.handleGestureEnd()
    });
    this.input.setDebug(this.debug);
  }

  destroy(): void {
    this.input?.destroy();
  }

  setDebug(enabled: boolean): void {
    this.debug = enabled;
    this.input?.setDebug(enabled);
  }

  getViewState(): PulseInputViewState {
    return {
      selectedNodeId: this.selectedNodeId,
      nearestNodeId: this.nearestNodeId,
      previewFromId: this.previewFromId,
      previewPoint: this.previewPoint ? { ...this.previewPoint } : undefined,
      liveGesture: this.liveGesture.map((point) => ({ ...point })),
      lastResult: this.lastResult
    };
  }

  getDebugInfo() {
    return this.input?.getDebugInfo();
  }

  clearSelection(): void {
    this.selectedNodeId = undefined;
    this.previewFromId = undefined;
    this.previewPoint = undefined;
    this.nearestNodeId = undefined;
    this.liveGesture = [];
    this.sim.selectNode(undefined);
  }

  private handleTap(point: GesturePoint): void {
    const snapshot = this.sim.getSnapshot();
    if (snapshot.phase !== 'build') {
      return;
    }
    const node = this.nearestNode(point, 76);
    if (!node) {
      this.clearSelection();
      this.lastResult = { ok: false, kind: 'invalid', message: 'NO NODE' };
      return;
    }
    if (this.selectedNodeId === undefined) {
      this.selectedNodeId = node.id;
      this.lastResult = this.sim.selectNode(node.id);
      return;
    }
    if (this.selectedNodeId === node.id) {
      this.clearSelection();
      return;
    }
    this.lastResult = this.sim.addLink(this.selectedNodeId, node.id);
    this.selectedNodeId = undefined;
  }

  private handleSwipe(gesture: SwipeGesture): void {
    const snapshot = this.sim.getSnapshot();
    if (snapshot.phase === 'build') {
      const startNode = this.nearestNode(gesture.start, 88);
      const endNode = this.nearestNode(gesture.end, 94);
      if (startNode && endNode) {
        this.lastResult = this.sim.addLink(startNode.id, endNode.id);
      } else {
        this.lastResult = { ok: false, kind: 'invalid', message: 'DRAG NODE TO NODE' };
      }
      this.selectedNodeId = undefined;
    } else if (snapshot.phase === 'pulse') {
      this.lastResult = this.sim.applyLens(gesture.points);
    }
    this.liveGesture = [];
    this.previewFromId = undefined;
    this.previewPoint = undefined;
  }

  private handlePreview(points: readonly GesturePoint[]): void {
    const latest = points[points.length - 1];
    if (!latest) {
      return;
    }
    this.liveGesture = points.map((point) => ({ x: point.x, y: point.y }));
    this.nearestNodeId = this.nearestNode(latest, 92)?.id;
    const first = points[0];
    if (this.sim.getSnapshot().phase === 'build' && first) {
      this.previewFromId = this.nearestNode(first, 88)?.id;
      this.previewPoint = { x: latest.x, y: latest.y };
    }
  }

  private handleGestureEnd(): void {
    this.liveGesture = [];
    this.previewFromId = undefined;
    this.previewPoint = undefined;
  }

  private nearestNode(point: WorldPoint, maxDistance: number): PulseNode | undefined {
    let best: PulseNode | undefined;
    let bestDistance = maxDistance * maxDistance;
    for (const node of this.sim.getNodes()) {
      const distance = distanceSquared(point.x, point.y, node.x, node.y);
      if (distance <= bestDistance) {
        best = node;
        bestDistance = distance;
      }
    }
    return best;
  }
}
```

src/game/pulse/PulseLevelGenerator.ts

```
import { BLACK_HOLE_X, BLACK_HOLE_Y, WORLD_HEIGHT, WORLD_WIDTH } from '../constants';
import { createSeededRandom } from '../rng';
import type { PulseLevel, PulseNode, PulseNodeType } from './PulseTypes';

export function getDailyPulseSeed(date = new Date()): string {
  const year = date.getUTCFullYear();
  const month = String(date.getUTCMonth() + 1).padStart(2, '0');
  const day = String(date.getUTCDate()).padStart(2, '0');
  return `daily-${year}-${month}-${day}`;
```

```
}

export function generatePulseLevel(seed: string): PulseLevel {
  const rng = createSeededRandom(`pulse-chain-${seed}`);
  const firstSeed = seed === 'daily-2026-05-30' || seed === 'tutorial' || seed === 'eh-pulse-alpha';
  const baseNodes = firstSeed ? tutorialNodes() : generatedNodes(rng);
  return {
    seed,
    nodes: baseNodes,
    sourceId: 1,
    linkBudget: 6,
    targetScore: 1800,
    targetSurvivalMs: 45000
  };
}

function tutorialNodes(): PulseNode[] {
  const specs: Array<[PulseNodeType, number, number, number, string]> = [
    ['source', 255, 1388, 0, 'SOURCE'],
    ['energy', 420, 1165, 1, 'ENERGY'],
    ['delay', 640, 1015, 1, 'DELAY'],
    ['splitter', 780, 760, 2, 'SPLIT'],
    ['energy', 525, 610, 2, 'ENERGY'],
    ['energy', 835, 1215, 2, 'ENERGY'],
    ['conduit', 310, 810, 2, 'CONDUIT'],
    ['conduit', 700, 1410, 2, 'CONDUIT'],
    ['delay', 250, 1080, 1, 'DELAY'],
    ['splitter', 910, 935, 2, 'SPLIT'],
    ['energy', 485, 1515, 2, 'ENERGY'],
    ['conduit', 805, 545, 2, 'CONDUIT']
  ];
  return specs.map(([type, x, y, ring, label], index) => makeNode(index + 1, type, x, y, ring, label));
}

function generatedNodes(rng: () => number): PulseNode[] {
  const nodeTypes: PulseNodeType[] = [
    'source',
    'energy',
    'delay',
    'splitter',
    'energy',
    'conduit',
    'conduit',
    'energy',
    'conduit',
    'delay',
    'splitter',
    'energy',
    'conduit'
  ];

  const nodes: PulseNode[] = [];
  for (let index = 0; index < nodeTypes.length; index += 1) {
    const type = nodeTypes[index];
    if (type === 'source') {
      nodes.push(makeNode(1, 'source', 230 + rng() * 130, 1320 + rng() * 190, 0, 'SOURCE'));
      continue;
    }
    const ring = index < 6 ? 1 : 2;
    const angle = -2.55 + index * 0.53 + (rng() - 0.5) * 0.22;
    const radiusX = ring === 1 ? 280 + rng() * 48 : 420 + rng() * 78;
    const radiusY = ring === 1 ? 410 + rng() * 46 : 570 + rng() * 84;
    const x = clampToPlayfield(BLACK_HOLE_X + Math.cos(angle) * radiusX);
    const y = clampToPlayfieldY(BLACK_HOLE_Y + Math.sin(angle) * radiusY + 120);
    nodes.push(makeNode(index + 1, type, x, y, ring, labelFor(type)));
  }
  return separateNodes(nodes);
}

function separateNodes(nodes: PulseNode[]): PulseNode[] {
  for (let pass = 0; pass < 5; pass += 1) {
    for (let a = 0; a < nodes.length; a += 1) {
      for (let b = a + 1; b < nodes.length; b += 1) {
        const left = nodes[a];
        const right = nodes[b];
        const dx = right.x - left.x;
        const dy = right.y - left.y;
        const distance = Math.max(1, Math.hypot(dx, dy));
        if (distance >= 132) {
          continue;
        }
        const push = (132 - distance) / 2;
        left.x = clampToPlayfield(left.x - (dx / distance) * push);
        left.y = clampToPlayfieldY(left.y - (dy / distance) * push);
        right.x = clampToPlayfield(right.x + (dx / distance) * push);
        right.y = clampToPlayfieldY(right.y + (dy / distance) * push);
      }
    }
  }
  return nodes;
}

function makeNode(id: number, type: PulseNodeType, x: number, y: number, ring: number, label: string): PulseNode {
  return {
    id,
    type,
    x,
    y,
    ring,
    label,
    radius: type === 'source' ? 54 : type === 'splitter' ? 50 : 46,
    activationMs: 0,
    scoreCooldownMs: 0
  };
}

function labelFor(type: PulseNodeType): string {
  if (type === 'energy') {
    return 'ENERGY';
  }
  if (type === 'delay') {
    return 'DELAY';
  }
  if (type === 'splitter') {
    return 'SPLIT';
  }
  return 'CONDUIT';
}

function clampToPlayfield(x: number): number {
  return Math.max(126, Math.min(WORLD_WIDTH - 126, x));
}

function clampToPlayfieldY(y: number): number {
  return Math.max(260, Math.min(WORLD_HEIGHT - 330, y));
}
```

src/game/pulse/PulseMode.ts

```
import { Application, type Renderer } from 'pixi.js';
import { WORLD_HEIGHT, WORLD_WIDTH } from '../constants';
import { FixedStepLoop } from '../FixedStepLoop';
import type { WorldPoint } from '../gestures';
import { clamp } from '../math';
import { createSharePoster, type PosterFrame } from '../posterizer';
import { submitScore } from '../scoreClient';
import { PulseInputController } from './PulseInputController';
import { getDailyPulseSeed } from './PulseLevelGenerator';
import { PulseRenderer } from './PulseRenderer';
import { PulseSimulation } from './PulseSimulation';
import type { PulseReplayPayload, PulseSnapshot } from './PulseTypes';

export interface PulseModeOptions {
  seed?: string;
  startedAt: number;
}

export class PulseMode {
  private readonly app = new Application<Renderer>();
  private readonly sim: PulseSimulation;
  private readonly loop: FixedStepLoop;
  private renderer?: PulseRenderer;
  private input?: PulseInputController;
  private scale = 1;
  private offsetX = 0;
  private offsetY = 0;
  private paused = false;
  private debug = false;
  private scoreSubmitted = false;
  private sampleCooldownMs = 0;
  private frameSamples: PosterFrame[] = [];

  constructor(
    private readonly root: HTMLElement,
    options: PulseModeOptions
  ) {
    const seed = options.seed ?? getDailyPulseSeed();
    this.sim = new PulseSimulation({ seed, startedAt: options.startedAt });
    this.loop = new FixedStepLoop(
      (dtMs) => this.step(dtMs),
      () => this.render()
    );
  }

  async start(): Promise<void> {
    await this.app.init({
      autoDensity: true,
      autoStart: false,
      backgroundAlpha: 0,
      clearBeforeRender: true,
      hello: false,
      preference: 'webgl',
      preserveDrawingBuffer: true,
      powerPreference: 'high-performance',
      resizeTo: this.root,
      resolution: Math.min(window.devicePixelRatio || 1, 2)
    });
    this.root.appendChild(this.app.canvas);
    this.renderer = new PulseRenderer(this.app.stage);
    this.input = new PulseInputController(this.app.canvas, this.sim, (clientX, clientY) => this.screenToWorld(clientX, clientY));
    this.input.setDebug(this.debug);
    this.input.start();
    window.addEventListener('resize', this.resize);
    this.resize();
    this.loop.start();
  }

  destroy(): void {
    this.loop.stop();
    this.input?.destroy();
    this.renderer?.destroy();
    window.removeEventListener('resize', this.resize);
    this.app.destroy(true, { children: true, texture: true });
  }

  restart(): void {
    this.sim.reset();
    this.scoreSubmitted = false;
    this.frameSamples = [];
    this.sampleCooldownMs = 0;
    this.input?.clearSelection();
    this.loop.resetClock();
  }

  setPaused(paused: boolean): void {
    this.paused = paused;
    if (!paused) {
      this.loop.resetClock();
    }
  }

  setInputDebug(enabled: boolean): void {
    this.debug = enabled;
    this.input?.setDebug(enabled);
  }

  getMode(): 'pulse-chain' {
    return 'pulse-chain';
  }

  getSnapshot(): PulseSnapshot {
    return this.sim.getSnapshot();
  }

  getReplayPayload(): PulseReplayPayload {
    return this.sim.getReplayPayload();
  }

  getNodes() {
    return this.sim.getNodes();
  }

  getLinks() {
    return this.sim.getLinks();
  }

  getPulses() {
    return this.sim.getPulses();
  }

  getLastInputResult() {
    return this.sim.getLastInputResult();
  }

  addLink(fromId: number, toId: number) {
    return this.sim.addLink(fromId, toId);
  }
```

```
  clearLinks() {
    return this.sim.clearLinks();
  }

  undo() {
    return this.sim.undo();
  }

  playPulse() {
    return this.sim.playPulse();
  }

  simulateLens(points: readonly WorldPoint[]) {
    return this.sim.applyLens(points.map((point, index) => ({ ...point, t: index * 16 })));
  }

  forceBuildPhase(): void {
    this.sim.forceBuildPhase();
  }

  forcePulsePhase(): void {
    this.sim.forcePulsePhase();
  }

  forceCollapse(): void {
    this.sim.forceCollapse();
  }

  forceEnd(): void {
    this.forceCollapse();
  }

  async exportPoster(): Promise<string> {
    this.sampleFrame('current');
    const snapshot = this.sim.getSnapshot();
    const frames = this.frameSamples.length >= 3 ? this.frameSamples : this.makeFallbackFrames();
    return createSharePoster(frames, {
      score: snapshot.score,
      survivalMs: snapshot.timeMs,
      seed: snapshot.seed,
      phase: 1
    });
  }

  private step(dtMs: number): void {
    if (this.paused) {
      return;
    }
    const wasEnded = this.sim.getSnapshot().ended;
    this.sim.step(dtMs);
    const snapshot = this.sim.getSnapshot();
    this.sampleCooldownMs -= dtMs;
    if (this.sampleCooldownMs <= 0 && !snapshot.ended) {
      this.sampleFrame(`t${snapshot.timeMs}`);
      this.sampleCooldownMs = 1300;
    }
    if (!wasEnded && snapshot.ended && !this.scoreSubmitted) {
      this.scoreSubmitted = true;
      this.saveBest(snapshot);
      void submitScore(this.sim.getReplayPayload());
    }
  }

  private render(): void {
    this.renderer?.render(this.sim.getSnapshot(), this.input?.getViewState() ?? {
      liveGesture: [],
      lastResult: this.sim.getLastInputResult()
    }, {
      scale: this.scale,
      offsetX: this.offsetX,
      offsetY: this.offsetY,
      debug: this.debug
    });
    this.app.render();
  }

  private readonly resize = (): void => {
    const rect = this.root.getBoundingClientRect();
    this.scale = Math.min(rect.width / WORLD_WIDTH, rect.height / WORLD_HEIGHT);
    const viewWidth = WORLD_WIDTH * this.scale;
    const viewHeight = WORLD_HEIGHT * this.scale;
    this.offsetX = (rect.width - viewWidth) / 2;
    this.offsetY = (rect.height - viewHeight) / 2;
  };

  private screenToWorld(clientX: number, clientY: number): WorldPoint {
    const rect = this.app.canvas.getBoundingClientRect();
    return {
      x: clamp((clientX - rect.left - this.offsetX) / this.scale, 0, WORLD_WIDTH),
      y: clamp((clientY - rect.top - this.offsetY) / this.scale, 0, WORLD_HEIGHT)
    };
  }

  private sampleFrame(label: string): void {
    try {
      this.frameSamples.push({ dataUrl: this.app.canvas.toDataURL('image/png', 0.78), label });
      if (this.frameSamples.length > 3) {
        this.frameSamples.shift();
      }
    } catch {
      this.frameSamples = this.makeFallbackFrames();
    }
  }

  private makeFallbackFrames(): PosterFrame[] {
    const snapshot = this.sim.getSnapshot();
    return [0, 1, 2].map((index) => ({
      dataUrl: this.makeMockFrame(snapshot, index),
      label: `pulse-${index}`
    }));
  }

  private makeMockFrame(snapshot: PulseSnapshot, index: number): string {
    const canvas = document.createElement('canvas');
    canvas.width = 540;
    canvas.height = 960;
    const context = canvas.getContext('2d');
    if (!context) {
      return '';
    }
    context.fillStyle = '#03040a';
    context.fillRect(0, 0, canvas.width, canvas.height);
    context.fillStyle = index === 0 ? '#16315c' : index === 1 ? '#45246d' : '#0f5b69';
    context.beginPath();
    context.arc(270, 410, 150, 0, Math.PI * 2);
    context.fill();
    context.fillStyle = '#000';
    context.beginPath();
    context.arc(270, 410, 72, 0, Math.PI * 2);
    context.fill();
    context.fillStyle = '#f7fbff';
    context.font = '800 52px system-ui';
    context.fillText(String(snapshot.score), 54, 824);
```

```
    return canvas.toDataURL('image/png', 0.78);
  }

  private saveBest(snapshot: PulseSnapshot): void {
    try {
      const bestScore = Number(localStorage.getItem('eventHorizon.bestScore') ?? 0);
      const bestSurvival = Number(localStorage.getItem('eventHorizon.bestSurvivalMs') ?? 0);
      if (snapshot.score > bestScore) {
        localStorage.setItem('eventHorizon.bestScore', String(snapshot.score));
      }
      if (snapshot.timeMs > bestSurvival) {
        localStorage.setItem('eventHorizon.bestSurvivalMs', String(snapshot.timeMs));
      }
    } catch {
      // Ignore storage failures in private or embedded browsers.
    }
  }

}
```

src/game/pulse/PulseRenderer.ts

```
import { Container, Graphics, Text, TextStyle } from 'pixi.js';
import { BLACK_HOLE_X, BLACK_HOLE_Y, MAX_ENERGY, WORLD_HEIGHT, WORLD_WIDTH } from '../constants';
import { clamp, formatTime } from '../math';
import { curvedLinkPath, pointOnQuadratic, resamplePath } from './PulseGeometry';
import type { PulseInputViewState } from './PulseInputController';
import type { PulseNode, PulseSnapshot } from './PulseTypes';
import type { WorldPoint } from '../gestures';

const NODE_COLORS = {
  source: 0x67f4ff,
  conduit: 0x9bb6ff,
  energy: 0x4dffbf,
  delay: 0xffd166,
  splitter: 0xd267ff
} as const;

export class PulseRenderer {
  private readonly world = new Container();
  private readonly background = new Graphics();
  private readonly linkLayer = new Graphics();
  private readonly tempLinkLayer = new Graphics();
  private readonly lensLayer = new Graphics();
  private readonly previewLayer = new Graphics();
  private readonly nodeLayer = new Graphics();
  private readonly pulseLayer = new Graphics();
  private readonly blackHole = new Graphics();
  private readonly hud = new Container();
  private readonly scoreText = new Text({
    text: '0',
    style: new TextStyle({ fill: '#f7fbff', fontFamily: 'Inter, system-ui, sans-serif', fontSize: 50, fontWeight: '900' })
  });
  private readonly metaText = new Text({
    text: '',
    style: new TextStyle({ fill: '#9fe7ff', fontFamily: 'Inter, system-ui, sans-serif', fontSize: 24, fontWeight: '800' })
  });
  private readonly hintText = new Text({
    text: '',
    style: new TextStyle({
      align: 'center',
      fill: '#ffffff',
      fontFamily: 'Inter, system-ui, sans-serif',
      fontSize: 34,
      fontWeight: '900',
      stroke: { color: '#180923', width: 5 }
    })
  });
  private readonly meter = new Graphics();
  private readonly debugText = new Text({
    text: '',
    style: new TextStyle({ fill: '#dff8ff', fontFamily: 'SFMono-Regular, Menlo, monospace', fontSize: 17, fontWeight: '600' })
  });
  private readonly endText = new Text({
    text: '',
    style: new TextStyle({
      align: 'center',
      fill: '#ffffff',
      fontFamily: 'Inter, system-ui, sans-serif',
      fontSize: 48,
      fontWeight: '900',
      stroke: { color: '#12051c', width: 6 }
    })
  });

  constructor(private readonly stage: Container) {
    this.stage.addChild(this.world);
    this.world.addChild(
      this.background,
      this.linkLayer,
      this.tempLinkLayer,
      this.blackHole,
      this.lensLayer,
      this.previewLayer,
      this.nodeLayer,
      this.pulseLayer,
      this.hintText,
      this.hud
    );
    this.hud.addChild(this.meter, this.scoreText, this.metaText, this.debugText, this.endText);
    this.hintText.anchor.set(0.5);
    this.hintText.position.set(WORLD_WIDTH / 2, 260);
    this.scoreText.position.set(68, 68);
    this.metaText.position.set(72, 130);
    this.debugText.position.set(72, 178);
    this.endText.anchor.set(0.5);
    this.endText.position.set(WORLD_WIDTH / 2, WORLD_HEIGHT * 0.52);
    this.drawBackground();
  }

  render(
    snapshot: PulseSnapshot,
    input: PulseInputViewState,
    options: { scale: number; offsetX: number; offsetY: number; debug: boolean }
  ): void {
    this.world.position.set(options.offsetX, options.offsetY);
    this.world.scale.set(options.scale);
    this.renderLinks(snapshot);
    this.renderBlackHole(snapshot);
    this.renderLenses(snapshot);
    this.renderPreview(snapshot, input);
    this.renderNodes(snapshot, input);
    this.renderPulses(snapshot);
    this.renderHud(snapshot, input, options.debug);
  }

  destroy(): void {
    this.stage.removeChild(this.world);
    this.world.destroy({ children: true });
  }
```

```
  private drawBackground(): void {
    this.background.clear();
    this.background.rect(0, 0, WORLD_WIDTH, WORLD_HEIGHT).fill(0x03040a);
    for (let index = 0; index < 260; index += 1) {
      const x = (index * 197.63) % WORLD_WIDTH;
      const y = (index * 311.19) % WORLD_HEIGHT;
      const depth = (index % 13) / 12;
      this.background.circle(x, y, 1.1 + depth * 2.3).fill({ color: 0xc8efff, alpha: 0.17 + depth * 0.35 });
    }
    for (let index = 0; index < 170; index += 1) {
      const t = index / 169;
      const angle = t * Math.PI * 8.6;
      const radius = 80 + t * 690;
      const x = BLACK_HOLE_X + Math.cos(angle) * radius;
      const y = BLACK_HOLE_Y + Math.sin(angle) * radius * 0.56;
      this.background.circle(x, y, 3 + t * 6).fill({ color: index % 2 === 0 ? 0x264e9a : 0x743a96, alpha: 0.05 + (1 - t) * 0.08 });
    }
  }

  private renderLinks(snapshot: PulseSnapshot): void {
    this.linkLayer.clear();
    this.tempLinkLayer.clear();
    for (const link of snapshot.links) {
      const from = findNode(snapshot.nodes, link.fromId);
      const to = findNode(snapshot.nodes, link.toId);
      if (!from || !to) {
        continue;
      }
      const layer = link.temporary ? this.tempLinkLayer : this.linkLayer;
      const alpha = link.temporary ? clamp(1 - link.ageMs / link.expiresMs, 0, 1) : 1;
      this.drawCurve(layer, curvedLinkPath(from, to, link.temporary ? -0.16 : 0.12), {
        glowColor: link.temporary ? 0xd267ff : 0x4dccff,
        coreColor: link.temporary ? 0xffffff : 0x9fe7ff,
        alpha,
        width: link.temporary ? 9 : 6
      });
      this.drawFlowDots(layer, from, to, snapshot.timeMs, link.temporary, alpha);
    }
  }

  private renderBlackHole(snapshot: PulseSnapshot): void {
    this.blackHole.clear();
    const pulse = Math.sin(snapshot.timeMs * 0.004) * 0.5 + 0.5;
    const collapse = 1 - snapshot.darkEnergy / MAX_ENERGY;
    const radius = 92 + collapse * 58 + (snapshot.phase === 'ended' && snapshot.collapsed ? 130 : 0);
    this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius * 2.2).fill({ color: 0x0b1426, alpha: 0.12 + collapse * 0.18 });
    this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius * 1.28).stroke({
      color: snapshot.phase === 'ended' && snapshot.stabilized ? 0x4dffbf : 0x6bcfff,
      alpha: 0.22 + collapse * 0.28,
      width: 7 + pulse * 6
    });
    this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius).fill({ color: 0x000000, alpha: 0.86 });
    this.blackHole.circle(BLACK_HOLE_X, BLACK_HOLE_Y, radius * 0.38).fill({ color: 0x03040a, alpha: 1 });
  }

  private renderLenses(snapshot: PulseSnapshot): void {
    this.lensLayer.clear();
    for (const lens of snapshot.lenses) {
      const alpha = clamp(1 - lens.ageMs / lens.durationMs, 0, 1);
      const smooth = resamplePath(lens.path, 44);
      this.drawCurve(this.lensLayer, smooth, {
        glowColor: lens.success ? 0xd267ff : 0xff6a83,
        coreColor: lens.success ? 0xffffff : 0xffb0c0,
        alpha,
        width: 8
      });
      const end = smooth[smooth.length - 1];
      this.lensLayer.circle(end.x, end.y, 15).stroke({ color: lens.success ? 0xffffff : 0xff6a83, alpha, width: 4 });
    }
  }

  private renderPreview(snapshot: PulseSnapshot, input: PulseInputViewState): void {
    this.previewLayer.clear();
    if (snapshot.phase === 'build' && input.previewFromId && input.previewPoint) {
      const from = findNode(snapshot.nodes, input.previewFromId);
      if (from) {
        this.drawCurve(this.previewLayer, curvedLinkPath(from, input.previewPoint, 0.08), {
          glowColor: 0xd267ff,
          coreColor: 0xffffff,
          alpha: 0.58,
          width: 5
        });
      }
    }
    if (snapshot.phase === 'pulse' && input.liveGesture.length > 1) {
      this.drawCurve(this.previewLayer, resamplePath(input.liveGesture, 36), {
        glowColor: 0xd267ff,
        coreColor: 0xffffff,
        alpha: 0.85,
        width: 7
      });
    }
  }

  private renderNodes(snapshot: PulseSnapshot, input: PulseInputViewState): void {
    this.nodeLayer.clear();
    for (const node of snapshot.nodes) {
      const selected = input.selectedNodeId === node.id;
      const nearest = input.nearestNodeId === node.id;
      const active = node.activationMs > 0;
      const color = NODE_COLORS[node.type];
      const halo = selected || nearest || active ? 0.56 : node.type === 'energy' || node.type === 'source' ? 0.34 : 0.22;
      this.nodeLayer.circle(node.x, node.y, node.radius + 26 + (active ? 18 : 0)).fill({ color, alpha: halo * 0.23 });
      this.nodeLayer.circle(node.x, node.y, node.radius + 12).stroke({ color, alpha: halo, width: selected ? 7 : 4 });
      if (node.type === 'splitter') {
        this.nodeLayer.regularPoly(node.x, node.y, node.radius, 3, -Math.PI / 2).fill({ color, alpha: 0.86 });
      } else if (node.type === 'delay') {
        this.nodeLayer.roundRect(node.x - node.radius * 0.74, node.y - node.radius * 0.74, node.radius * 1.48, node.radius * 1.48, 12).fill({ color, alpha: 0.88 });
      } else {
        this.nodeLayer.circle(node.x, node.y, node.radius).fill({ color, alpha: 0.88 });
      }
      this.nodeLayer.circle(node.x, node.y, node.radius * 0.45).fill({ color: 0xffffff, alpha: active ? 0.82 : 0.36 });
    }
  }

  private renderPulses(snapshot: PulseSnapshot): void {
    this.pulseLayer.clear();
    for (const pulse of snapshot.pulses) {
      const point = pulsePoint(snapshot, pulse.currentNodeId, pulse.nextNodeId, pulse.progress);
      if (!point) {
        continue;
      }
      this.pulseLayer.circle(point.x, point.y, 30).fill({ color: 0x67f4ff, alpha: 0.18 });
      this.pulseLayer.circle(point.x, point.y, 15).fill({ color: 0xd267ff, alpha: 0.72 });
      this.pulseLayer.circle(point.x, point.y, 7).fill({ color: 0xffffff, alpha: 0.95 });
    }
  }

  private renderHud(snapshot: PulseSnapshot, input: PulseInputViewState, debug: boolean): void {
    this.scoreText.text = String(snapshot.score);
    this.metaText.text = `x${snapshot.multiplier}  LINKS ${snapshot.linksUsed}/${snapshot.linkBudget}  LENS ${snapshot.lensCharges}/2  BEST ${Math.max(snapshot.score,
Number(localStorage.getItem('eventHorizon.bestScore') ?? 0))}`;
```

```
    this.hintText.text = snapshot.tutorialHint;
    this.hintText.visible = snapshot.phase !== 'ended';
    this.meter.clear();
    const meterWidth = WORLD_WIDTH - 156;
    const fill = meterWidth * clamp(snapshot.darkEnergy / MAX_ENERGY, 0, 1);
    this.meter.roundRect(78, WORLD_HEIGHT - 136, meterWidth, 40, 8).fill({ color: 0x061120, alpha: 0.9 });
    this.meter.roundRect(78, WORLD_HEIGHT - 136, meterWidth, 40, 8).stroke({ color: 0x78f2ff, alpha: 0.35, width: 2 });
    this.meter.roundRect(86, WORLD_HEIGHT - 126, fill, 20, 6).fill({ color: snapshot.darkEnergy < 25 ? 0xff5d73 : 0x67f4ff, alpha: 0.96 });
    this.meter.roundRect(86, WORLD_HEIGHT - 172, 220, 28, 4).fill({ color: 0x03040a, alpha: 0.46 });
    this.meter.roundRect(0, 0, 0, 0, 0);
    this.endText.visible = snapshot.phase === 'ended';
    this.endText.text = snapshot.phase === 'ended'
      ? `${snapshot.stabilized ? 'SECTOR STABILIZED' : 'GALAXY COLLAPSED'}\n${snapshot.score}  •  ${formatTime(snapshot.timeMs)}\nSEED ${snapshot.seed}`
      : '';
    this.debugText.visible = debug;
    if (debug) {
      this.debugText.text = [
        `phase: ${snapshot.phase}`,
        `selected: ${input.selectedNodeId ?? '--'} nearest: ${input.nearestNodeId ?? '--'}`,
        `links: ${snapshot.linksUsed}/${snapshot.linkBudget} pulses: ${snapshot.pulses.length}`,
        `last: ${snapshot.lastInputResult.message}`,
        `hash: ${snapshot.stepHash}`
      ].join('\n');
    }
  }

  private drawCurve(
    graphics: Graphics,
    points: readonly WorldPoint[],
    options: { glowColor: number; coreColor: number; alpha: number; width: number }
  ): void {
    if (points.length < 2) {
      return;
    }
    drawSmooth(graphics, points);
    graphics.stroke({ color: options.glowColor, alpha: options.alpha * 0.28, width: options.width * 3.4 });
    drawSmooth(graphics, points);
    graphics.stroke({ color: options.coreColor, alpha: options.alpha, width: options.width });
  }

  private drawFlowDots(
    graphics: Graphics,
    from: PulseNode,
    to: PulseNode,
    timeMs: number,
    temporary: boolean,
    alpha: number
  ): void {
    const path = curvedLinkPath(from, to, temporary ? -0.16 : 0.12);
    for (let index = 0; index < 3; index += 1) {
      const t = ((timeMs * 0.00018 + index / 3) % 1 + 1) % 1;
      const point = pointOnQuadratic(path[0], path[1], path[2], t);
      graphics.circle(point.x, point.y, temporary ? 6 : 4).fill({ color: temporary ? 0xffffff : 0x9fe7ff, alpha: alpha * 0.66 });
    }
  }
}

function drawSmooth(graphics: Graphics, points: readonly WorldPoint[]): void {
  graphics.moveTo(points[0].x, points[0].y);
  for (let index = 1; index < points.length - 1; index += 1) {
    const point = points[index];
    const next = points[index + 1];
    graphics.quadraticCurveTo(point.x, point.y, (point.x + next.x) / 2, (point.y + next.y) / 2);
  }
  const last = points[points.length - 1];
  graphics.lineTo(last.x, last.y);
}

function findNode(nodes: readonly PulseNode[], id: number): PulseNode | undefined {
  return nodes.find((node) => node.id === id);
}

function pulsePoint(snapshot: PulseSnapshot, fromId: number, toId: number | undefined, progress: number): WorldPoint | undefined {
  const from = findNode(snapshot.nodes, fromId);
  if (!from) {
    return undefined;
  }
  if (toId === undefined) {
    return from;
  }
  const to = findNode(snapshot.nodes, toId);
  if (!to) {
    return from;
  }
  const path = curvedLinkPath(from, to, 0.12);
  return pointOnQuadratic(path[0], path[1], path[2], progress);
}
```

src/game/pulse/PulseSimulation.ts

```
import { INITIAL_ENERGY, MAX_ENERGY } from '../constants';
import { clamp } from '../math';
import { quantizeGesturePath, type GesturePathPoint, type WorldPoint } from '../gestures';
import { nearestTwoNodesToPath } from './PulseGeometry';
import { generatePulseLevel } from './PulseLevelGenerator';
import type {
  BuildInput,
  HorizonLens,
  LiveInput,
  PulseGamePhase,
  PulseInputResult,
  PulseLevel,
  PulseLink,
  PulseNode,
  PulseReplayPayload,
  PulseResult,
  PulseSnapshot,
  PulseState
} from './PulseTypes';

const PULSE_SPEED = 460;
const LINK_TRAVERSAL_SCORE = 10;
const ENERGY_NODE_SCORE = 100;
const DELAY_NODE_SCORE = 25;
const SPLITTER_NODE_SCORE = 75;
const LENS_DURATION_MS = 1200;
const LENS_MAX_CHARGES = 2;
const ENERGY_DRAIN_PER_SECOND = 0.52;

export interface PulseSimulationOptions {
  seed: string;
  startedAt: number;
}

export class PulseSimulation {
  readonly seed: string;
  readonly startedAt: number;
  private level: PulseLevel;
  private phase: PulseGamePhase = 'build';
  private timeMs = 0;
```

```
  private score = 0;
  private darkEnergy = INITIAL_ENERGY;
  private multiplier = 1;
  private maxMultiplier = 1;
  private chainLength = 0;
  private loopsCompleted = 0;
  private nextLinkId = 1;
  private nextPulseId = 1;
  private nextLensId = 1;
  private endReason: PulseSnapshot['endReason'];
  private collapsed = false;
  private stabilized = false;
  private buildInputs: BuildInput[] = [];
  private liveInputs: LiveInput[] = [];
  private links: PulseLink[] = [];
  private pulses: PulseState[] = [];
  private lenses: HorizonLens[] = [];
  private lastInputResult: PulseInputResult = { ok: true, kind: 'none', message: 'CONNECT TWO NODES' };
  private selectedNodeId: number | undefined;
  private stepHash = '00000000';
  private lensCharges = LENS_MAX_CHARGES;

  constructor(options: PulseSimulationOptions) {
    this.seed = options.seed;
    this.startedAt = options.startedAt;
    this.level = generatePulseLevel(options.seed);
    this.updateHash();
  }

  reset(seed = this.seed): void {
    this.level = generatePulseLevel(seed);
    this.phase = 'build';
    this.timeMs = 0;
    this.score = 0;
    this.darkEnergy = INITIAL_ENERGY;
    this.multiplier = 1;
    this.maxMultiplier = 1;
    this.chainLength = 0;
    this.loopsCompleted = 0;
    this.nextLinkId = 1;
    this.nextPulseId = 1;
    this.nextLensId = 1;
    this.endReason = undefined;
    this.collapsed = false;
    this.stabilized = false;
    this.buildInputs = [];
    this.liveInputs = [];
    this.links = [];
    this.pulses = [];
    this.lenses = [];
    this.selectedNodeId = undefined;
    this.lensCharges = LENS_MAX_CHARGES;
    this.lastInputResult = { ok: true, kind: 'none', message: 'CONNECT TWO NODES' };
    this.updateHash();
  }

  step(dtMs: number): void {
    if (this.phase === 'ended') {
      this.timeMs += dtMs;
      this.updateTimedVisuals(dtMs);
      this.updateHash();
      return;
    }

    this.timeMs += dtMs;
    this.updateTimedVisuals(dtMs);

    if (this.phase === 'pulse') {
      this.darkEnergy = clamp(this.darkEnergy - (dtMs / 1000) * ENERGY_DRAIN_PER_SECOND, 0, MAX_ENERGY);
      this.updatePulses(dtMs);
      this.expireTemporaryLinks(dtMs);
      if (this.darkEnergy <= 0) {
        this.endRun('collapsed');
      } else if (this.score >= this.level.targetScore || this.timeMs >= this.level.targetSurvivalMs) {
        this.endRun('stabilized');
      } else if (this.pulses.every((pulse) => !pulse.alive) && this.timeMs > 500) {
        this.darkEnergy = clamp(this.darkEnergy - 0.9, 0, MAX_ENERGY);
        if (this.darkEnergy <= 0) {
          this.endRun('collapsed');
        } else {
          this.endRun('pulse-died');
        }
      }
    }
    this.updateHash();
  }

  selectNode(nodeId: number | undefined): PulseInputResult {
    this.selectedNodeId = nodeId;
    this.lastInputResult = nodeId
      ? { ok: true, kind: 'select', message: 'NODE SELECTED', fromId: nodeId }
      : { ok: true, kind: 'none', message: 'SELECT A NODE' };
    return this.lastInputResult;
  }

  addLink(fromId: number, toId: number, record = true): PulseInputResult {
    const validation = this.validateLink(fromId, toId, false);
    if (!validation.ok) {
      this.lastInputResult = validation;
      return validation;
    }
    const link: PulseLink = {
      id: this.nextLinkId,
      fromId,
      toId,
      temporary: false,
      ageMs: 0,
      expiresMs: 0
    };
    this.nextLinkId += 1;
    this.links.push(link);
    if (record) {
      this.buildInputs.push({ t: Math.round(this.timeMs), kind: 'link', fromId, toId });
    }
    this.selectedNodeId = undefined;
    this.lastInputResult = { ok: true, kind: 'link', message: 'GRAVITATIONAL LINK', fromId, toId };
    this.updateHash();
    return this.lastInputResult;
  }

  undo(): PulseInputResult {
    const index = [...this.links].map((link, linkIndex) => ({ link, linkIndex })).reverse().find((entry) => !entry.link.temporary);
    if (!index) {
      this.lastInputResult = { ok: false, kind: 'invalid', message: 'NO LINK TO UNDO' };
      return this.lastInputResult;
    }
    this.links.splice(index.linkIndex, 1);
    this.buildInputs.push({ t: Math.round(this.timeMs), kind: 'undo' });
    this.lastInputResult = { ok: true, kind: 'undo', message: 'LINK UNDONE' };
    this.updateHash();
    return this.lastInputResult;
  }
```

```
  clearLinks(): PulseInputResult {
    this.links = this.links.filter((link) => link.temporary);
    this.buildInputs.push({ t: Math.round(this.timeMs), kind: 'clear' });
    this.selectedNodeId = undefined;
    this.lastInputResult = { ok: true, kind: 'clear', message: 'LINKS CLEARED' };
    this.updateHash();
    return this.lastInputResult;
  }

  playPulse(record = true): PulseInputResult {
    if (this.phase !== 'build') {
      this.lastInputResult = { ok: false, kind: 'invalid', message: 'PULSE ALREADY RUNNING' };
      return this.lastInputResult;
    }
    const outgoing = this.outgoingLinks(this.level.sourceId);
    if (outgoing.length === 0) {
      this.lastInputResult = { ok: false, kind: 'invalid', message: 'CONNECT SOURCE FIRST' };
      return this.lastInputResult;
    }
    this.phase = 'pulse';
    this.timeMs = 0;
    this.emitFromNode(this.level.sourceId, undefined, 0, []);
    if (record) {
      this.buildInputs.push({ t: 0, kind: 'play' });
    }
    this.lastInputResult = { ok: true, kind: 'play', message: 'STABILIZING PULSE' };
    this.updateHash();
    return this.lastInputResult;
  }

  applyLens(points: readonly GesturePathPoint[]): PulseInputResult {
    const path = quantizeGesturePath(points, 24);
    const lensPath = path.map((point) => ({ x: point.x, y: point.y }));
    if (this.phase !== 'pulse') {
      this.lastInputResult = { ok: false, kind: 'invalid', message: 'LENS ONLY DURING PLAYBACK' };
      return this.lastInputResult;
    }
    if (this.lensCharges <= 0) {
      this.lastInputResult = { ok: false, kind: 'lens', message: 'LENS RECHARGING' };
      return this.lastInputResult;
    }

    const anchors = nearestTwoNodesToPath(this.level.nodes, lensPath, 132);
    if (!anchors) {
      this.createLens(lensPath, false);
      this.liveInputs.push({ t: Math.round(this.timeMs), kind: 'lens', path, success: false });
      this.lastInputResult = { ok: false, kind: 'lens', message: 'NO ANCHOR' };
      return this.lastInputResult;
    }

    const [from, to] = anchors;
    const validation = this.validateLink(from.id, to.id, true);
    if (!validation.ok) {
      this.createLens(lensPath, false);
      this.liveInputs.push({ t: Math.round(this.timeMs), kind: 'lens', path, fromId: from.id, toId: to.id, success: false });
      this.lastInputResult = { ok: false, kind: 'lens', message: 'NO ANCHOR', fromId: from.id, toId: to.id };
      return this.lastInputResult;
    }

    this.links.push({
      id: this.nextLinkId,
      fromId: from.id,
      toId: to.id,
      temporary: true,
      ageMs: 0,
      expiresMs: LENS_DURATION_MS
    });
    this.nextLinkId += 1;
    this.lensCharges = Math.max(0, this.lensCharges - 1);
    this.createLens(lensPath, true, from.id, to.id);
    this.liveInputs.push({ t: Math.round(this.timeMs), kind: 'lens', path, fromId: from.id, toId: to.id, success: true });
    this.lastInputResult = { ok: true, kind: 'lens', message: 'BRIDGE CREATED', fromId: from.id, toId: to.id };
    this.updateHash();
    return this.lastInputResult;
  }

  forceBuildPhase(): void {
    this.phase = 'build';
    this.pulses = [];
    this.endReason = undefined;
    this.collapsed = false;
    this.stabilized = false;
    this.darkEnergy = Math.max(this.darkEnergy, 45);
    this.updateHash();
  }

  forcePulsePhase(): void {
    if (this.phase === 'build') {
      void this.playPulse(false);
    }
  }

  forceCollapse(): void {
    this.endRun('collapsed');
  }

  getNodes(): readonly PulseNode[] {
    return this.level.nodes;
  }

  getLinks(): readonly PulseLink[] {
    return this.links;
  }

  getPulses(): readonly PulseState[] {
    return this.pulses;
  }

  getLastInputResult(): PulseInputResult {
    return { ...this.lastInputResult };
  }

  getReplayPayload(): PulseReplayPayload {
    return {
      version: 1,
      mode: 'pulse-chain',
      seed: this.seed,
      startedAt: this.startedAt,
      buildInputs: this.buildInputs.map((input) => ({ ...input })),
      liveInputs: this.liveInputs.map((input) => ({
        ...input,
        path: input.path.map((point) => ({ ...point }))
      })),
      result: this.getResult(),
      stepHash: this.stepHash
    };
  }

  getSnapshot(): PulseSnapshot {
    return {
      mode: 'pulse-chain',
      seed: this.seed,
      phase: this.phase,
```

```
      timeMs: Math.round(this.timeMs),
      score: this.score,
      darkEnergy: this.darkEnergy,
      collapseMeter: this.darkEnergy,
      multiplier: this.multiplier,
      maxMultiplier: this.maxMultiplier,
      chainLength: this.chainLength,
      loopsCompleted: this.loopsCompleted,
      linkBudget: this.level.linkBudget,
      linksUsed: this.links.filter((link) => !link.temporary).length,
      lensCharges: this.lensCharges,
      targetScore: this.level.targetScore,
      targetSurvivalMs: this.level.targetSurvivalMs,
      ended: this.phase === 'ended',
      endReason: this.endReason,
      collapsed: this.collapsed,
      stabilized: this.stabilized,
      nodes: this.level.nodes,
      links: this.links,
      pulses: this.pulses,
      lenses: this.lenses,
      selectedNodeId: this.selectedNodeId,
      tutorialHint: this.tutorialHint(),
      lastInputResult: this.lastInputResult,
      stepHash: this.stepHash
    };
  }

  private validateLink(fromId: number, toId: number, temporary: boolean): PulseInputResult {
    if (this.phase !== 'build' && !temporary) {
      return { ok: false, kind: 'invalid', message: 'LINKS LOCKED' };
    }
    const from = this.nodeById(fromId);
    const to = this.nodeById(toId);
    if (!from || !to) {
      return { ok: false, kind: 'invalid', message: 'NO ANCHOR', fromId, toId };
    }
    if (fromId === toId) {
      return { ok: false, kind: 'invalid', message: 'NO SELF LINK', fromId, toId };
    }
    if (this.links.some((link) => link.fromId === fromId && link.toId === toId)) {
      return { ok: false, kind: 'invalid', message: 'LINK EXISTS', fromId, toId };
    }
    if (!temporary && this.links.filter((link) => !link.temporary).length >= this.level.linkBudget) {
      return { ok: false, kind: 'invalid', message: 'LINK LIMIT', fromId, toId };
    }
    const maxOutgoing = from.type === 'splitter' ? 3 : from.type === 'source' ? 2 : 1;
    if (this.outgoingLinks(fromId).filter((link) => !link.temporary || temporary).length >= maxOutgoing) {
      return { ok: false, kind: 'invalid', message: 'NODE OUTPUT FULL', fromId, toId };
    }
    return { ok: true, kind: 'link', message: 'VALID', fromId, toId };
  }

  private updatePulses(dtMs: number): void {
    for (const pulse of this.pulses) {
      if (!pulse.alive) {
        continue;
      }
      pulse.ageMs += dtMs;
      if (pulse.delayMs > 0) {
        pulse.delayMs = Math.max(0, pulse.delayMs - dtMs);
        continue;
      }
      if (pulse.nextNodeId === undefined) {
        this.continuePulse(pulse);
        continue;
      }
      const from = this.nodeById(pulse.currentNodeId);
      const to = this.nodeById(pulse.nextNodeId);
      if (!from || !to) {
        this.killPulse(pulse);
        continue;
      }
      const length = Math.max(1, Math.hypot(to.x - from.x, to.y - from.y));
      pulse.progress += (pulse.speed * (dtMs / 1000)) / length;
      if (pulse.progress >= 1) {
        this.arriveAtNode(pulse, to.id);
      }
    }
    this.pulses = this.pulses.filter((pulse) => pulse.alive);
  }

  private arriveAtNode(pulse: PulseState, nodeId: number): void {
    const node = this.nodeById(nodeId);
    if (!node) {
      this.killPulse(pulse);
      return;
    }
    pulse.previousNodeId = pulse.currentNodeId;
    pulse.currentNodeId = nodeId;
    pulse.nextNodeId = undefined;
    pulse.progress = 0;
    pulse.comboChainLength += 1;
    pulse.visitedNodeIds.push(nodeId);
    this.chainLength = Math.max(this.chainLength, pulse.comboChainLength);
    this.multiplier = multiplierForChain(pulse.comboChainLength);
    this.maxMultiplier = Math.max(this.maxMultiplier, this.multiplier);
    node.activationMs = 520;
    this.addScore(LINK_TRAVERSAL_SCORE, 0.18);

    const firstRepeatedIndex = pulse.visitedNodeIds.indexOf(nodeId);
    if (firstRepeatedIndex >= 0 && firstRepeatedIndex < pulse.visitedNodeIds.length - 1) {
      const loopLength = pulse.visitedNodeIds.length - 1 - firstRepeatedIndex;
      if (loopLength >= 4) {
        this.loopsCompleted += 1;
        this.addScore(140 * this.loopsCompleted, 1.2);
        this.multiplier = Math.max(this.multiplier, Math.min(12, this.multiplier + this.loopsCompleted));
        this.maxMultiplier = Math.max(this.maxMultiplier, this.multiplier);
        this.lensCharges = Math.min(LENS_MAX_CHARGES, this.lensCharges + 1);
      }
    }

    if (node.type === 'energy') {
      const fresh = node.scoreCooldownMs <= 0;
      this.addScore(fresh ? ENERGY_NODE_SCORE : 25, fresh ? 6.2 : 1.4);
      node.scoreCooldownMs = fresh ? 2600 : node.scoreCooldownMs;
      this.lensCharges = Math.min(LENS_MAX_CHARGES, this.lensCharges + 1);
    } else if (node.type === 'delay') {
      this.addScore(DELAY_NODE_SCORE, 1.1);
      pulse.delayMs = 620;
      return;
    } else if (node.type === 'splitter') {
      this.addScore(SPLITTER_NODE_SCORE, 1.8);
    }

    this.continuePulse(pulse);
  }

  private continuePulse(pulse: PulseState): void {
    const outgoing = this.outgoingLinks(pulse.currentNodeId);
    if (outgoing.length === 0) {
      this.killPulse(pulse);
      return;
```

```
    }

    if (this.nodeById(pulse.currentNodeId)?.type === 'splitter' && outgoing.length > 1) {
      for (const link of outgoing) {
        this.spawnPulse(pulse.currentNodeId, link.toId, pulse.previousNodeId, pulse.comboChainLength, pulse.visitedNodeIds);
      }
      pulse.alive = false;
      return;
    }

    const preferred = outgoing.find((link) => link.toId !== pulse.previousNodeId) ?? outgoing[0];
    pulse.nextNodeId = preferred.toId;
    pulse.progress = 0;
  }

  private emitFromNode(nodeId: number, previousNodeId: number | undefined, combo: number, visited: number[]): void {
    const outgoing = this.outgoingLinks(nodeId);
    if (outgoing.length === 0) {
      return;
    }
    for (const link of outgoing) {
      this.spawnPulse(nodeId, link.toId, previousNodeId, combo, visited.length > 0 ? visited : [nodeId]);
    }
  }

  private spawnPulse(
    currentNodeId: number,
    nextNodeId: number,
    previousNodeId: number | undefined,
    comboChainLength: number,
    visitedNodeIds: number[]
  ): void {
    this.pulses.push({
      id: this.nextPulseId,
      currentNodeId,
      previousNodeId,
      nextNodeId,
      progress: 0,
      speed: PULSE_SPEED,
      ageMs: 0,
      energy: 1,
      comboChainLength,
      delayMs: 0,
      alive: true,
      visitedNodeIds: [...visitedNodeIds]
    });
    this.nextPulseId += 1;
  }

  private killPulse(pulse: PulseState): void {
    pulse.alive = false;
    this.darkEnergy = clamp(this.darkEnergy - 5.6, 0, MAX_ENERGY);
    this.multiplier = 1;
    this.lastInputResult = { ok: false, kind: 'invalid', message: 'DEAD END' };
  }

  private addScore(base: number, energyGain: number): void {
    this.score += Math.round(base * this.multiplier);
    this.darkEnergy = clamp(this.darkEnergy + energyGain, 0, MAX_ENERGY);
  }

  private outgoingLinks(nodeId: number): PulseLink[] {
    return this.links.filter((link) => link.fromId === nodeId);
  }

  private nodeById(nodeId: number): PulseNode | undefined {
    return this.level.nodes.find((node) => node.id === nodeId);
  }

  private updateTimedVisuals(dtMs: number): void {
    for (const node of this.level.nodes) {
      node.activationMs = Math.max(0, node.activationMs - dtMs);
      node.scoreCooldownMs = Math.max(0, node.scoreCooldownMs - dtMs);
    }
    for (let index = this.lenses.length - 1; index >= 0; index -= 1) {
      const lens = this.lenses[index];
      lens.ageMs += dtMs;
      if (lens.ageMs >= lens.durationMs) {
        this.lenses.splice(index, 1);
      }
    }
  }

  private expireTemporaryLinks(dtMs: number): void {
    for (let index = this.links.length - 1; index >= 0; index -= 1) {
      const link = this.links[index];
      if (!link.temporary) {
        continue;
      }
      link.ageMs += dtMs;
      if (link.ageMs >= link.expiresMs) {
        this.links.splice(index, 1);
      }
    }
  }

  private createLens(path: WorldPoint[], success: boolean, fromId?: number, toId?: number): void {
    this.lenses.push({
      id: this.nextLensId,
      path,
      fromId,
      toId,
      ageMs: 0,
      durationMs: LENS_DURATION_MS,
      success,
      message: success ? 'BRIDGE CREATED' : 'NO ANCHOR'
    });
    this.nextLensId += 1;
  }

  private endRun(reason: Exclude<PulseSnapshot['endReason'], undefined>): void {
    if (this.phase === 'ended') {
      return;
    }
    this.phase = 'ended';
    this.endReason = reason;
    this.collapsed = reason === 'collapsed';
    this.stabilized = reason === 'stabilized';
    if (this.stabilized) {
      this.addScore(350 + (this.level.linkBudget - this.links.filter((link) => !link.temporary).length) * 80, 0);
    }
    this.pulses = [];
    this.updateHash();
  }

  private getResult(): PulseResult {
    return {
      score: this.score,
      survivalMs: Math.round(this.timeMs),
      maxMultiplier: this.maxMultiplier,
      loopsCompleted: this.loopsCompleted,
      linksUsed: this.links.filter((link) => !link.temporary).length,
      stabilized: this.stabilized,
```

```
      collapsed: this.collapsed
    };
  }

  private tutorialHint(): string {
    if (this.phase === 'build') {
      if (this.links.filter((link) => !link.temporary).length === 0) {
        return 'CONNECT TWO NODES';
      }
      return 'PRESS PLAY';
    }
    if (this.phase === 'pulse' && this.liveInputs.length === 0) {
      return 'SWIPE TO CREATE A HORIZON LENS';
    }
    return this.phase === 'ended' ? (this.stabilized ? 'SECTOR STABILIZED' : 'GALAXY COLLAPSED') : 'WATCH THE PULSE';
  }

  private updateHash(): void {
    const data = [
      this.phase,
      Math.round(this.timeMs),
      this.score,
      Math.round(this.darkEnergy * 10),
      this.multiplier,
      this.links.map((link) => `${link.fromId}>${link.toId}:${link.temporary ? Math.round(link.expiresMs - link.ageMs) : 0}`).join('|'),
      this.pulses.map((pulse) => `${pulse.currentNodeId}>${pulse.nextNodeId ?? 0}:${Math.round(pulse.progress * 1000)}:${pulse.delayMs}`).join('|')
    ].join(';');
    let hash = 2166136261;
    for (let index = 0; index < data.length; index += 1) {
      hash ^= data.charCodeAt(index);
      hash = Math.imul(hash, 16777619);
    }
    this.stepHash = (hash >>> 0).toString(16).padStart(8, '0');
  }
}

function multiplierForChain(chainLength: number): number {
  if (chainLength >= 15) {
    return 8;
  }
  if (chainLength >= 10) {
    return 5;
  }
  if (chainLength >= 7) {
    return 3;
  }
  if (chainLength >= 4) {
    return 2;
  }
  return 1;
}
```

src/game/pulse/PulseTypes.ts

```
import type { WorldPoint } from '../gestures';

export type PulseGamePhase = 'build' | 'pulse' | 'ended';
export type PulseEndReason = 'collapsed' | 'stabilized' | 'pulse-died' | 'manual';
export type PulseNodeType = 'source' | 'conduit' | 'energy' | 'delay' | 'splitter';

export interface PulseNode extends WorldPoint {
  id: number;
  type: PulseNodeType;
  radius: number;
  ring: number;
  label: string;
  activationMs: number;
  scoreCooldownMs: number;
}

export interface PulseLink {
  id: number;
  fromId: number;
  toId: number;
  temporary: boolean;
  ageMs: number;
  expiresMs: number;
}

export interface PulseState {
  id: number;
  currentNodeId: number;
  previousNodeId?: number;
  nextNodeId?: number;
  progress: number;
  speed: number;
  ageMs: number;
  energy: number;
  comboChainLength: number;
  delayMs: number;
  alive: boolean;
  visitedNodeIds: number[];
}

export interface HorizonLens {
  id: number;
  path: WorldPoint[];
  fromId?: number;
  toId?: number;
  ageMs: number;
  durationMs: number;
  success: boolean;
  message: 'HORIZON LENS' | 'BRIDGE CREATED' | 'NO ANCHOR';
}

export interface PulseLevel {
  seed: string;
  nodes: PulseNode[];
  sourceId: number;
  linkBudget: number;
  targetScore: number;
  targetSurvivalMs: number;
}

export type BuildInput =
  | { t: number; kind: 'link'; fromId: number; toId: number }
  | { t: number; kind: 'undo' }
  | { t: number; kind: 'clear' }
  | { t: number; kind: 'play' };

export interface LiveInput {
  t: number;
  kind: 'lens';
  path: { x: number; y: number; t: number }[];
  fromId?: number;
  toId?: number;
  success: boolean;
}

export interface PulseResult {
```

```
  score: number;
  survivalMs: number;
  maxMultiplier: number;
  loopsCompleted: number;
  linksUsed: number;
  stabilized: boolean;
  collapsed: boolean;
}

export interface PulseReplayPayload {
  version: 1;
  mode: 'pulse-chain';
  seed: string;
  startedAt: number;
  buildInputs: BuildInput[];
  liveInputs: LiveInput[];
  result: PulseResult;
  stepHash: string;
}

export interface PulseInputResult {
  ok: boolean;
  kind: 'select' | 'link' | 'undo' | 'clear' | 'play' | 'lens' | 'invalid' | 'none';
  message: string;
  fromId?: number;
  toId?: number;
}

export interface PulseSnapshot {
  mode: 'pulse-chain';
  seed: string;
  phase: PulseGamePhase;
  timeMs: number;
  score: number;
  darkEnergy: number;
  collapseMeter: number;
  multiplier: number;
  maxMultiplier: number;
  chainLength: number;
  loopsCompleted: number;
  linkBudget: number;
  linksUsed: number;
  lensCharges: number;
  targetScore: number;
  targetSurvivalMs: number;
  ended: boolean;
  endReason?: PulseEndReason;
  collapsed: boolean;
  stabilized: boolean;
  nodes: readonly PulseNode[];
  links: readonly PulseLink[];
  pulses: readonly PulseState[];
  lenses: readonly HorizonLens[];
  selectedNodeId?: number;
  tutorialHint: string;
  lastInputResult: PulseInputResult;
  stepHash: string;
}
```

src/game/scoreClient.ts

```
import { SCORE_ENDPOINT } from './constants';
import type { PulseReplayPayload } from './pulse/PulseTypes';
import type { ReplayPayload } from './types';

export interface ScoreSubmitResult {
  ok: boolean;
  status: number;
  endpoint: string;
}

export async function submitScore(
  payload: ReplayPayload | PulseReplayPayload,
  endpoint = SCORE_ENDPOINT
): Promise<ScoreSubmitResult> {
  try {
    const response = await fetch(endpoint, {
      method: 'POST',
      headers: {
        'content-type': 'application/json'
      },
      body: JSON.stringify(payload),
      keepalive: true
    });
    return {
      ok: response.ok,
      status: response.status,
      endpoint
    };
  } catch {
    return {
      ok: false,
      status: 0,
      endpoint
    };
  }
}
```

src/main.ts

```
import './styles.css';
import { EventHorizonGame } from './game/EventHorizonGame';
import { PulseMode } from './game/pulse/PulseMode';
import { getDailyPulseSeed } from './game/pulse/PulseLevelGenerator';

interface EventHorizonRuntime {
  start: () => Promise<void>;
  restart: () => void;
  setPaused: (paused: boolean) => void;
  setInputDebug: (enabled: boolean) => void;
  exportPoster: () => Promise<string>;
  forceEnd: () => void;
  getSnapshot: () => unknown;
  getReplayPayload: () => unknown;
  destroy?: () => void;
}

const root = document.querySelector<HTMLDivElement>('#game-root');
const restartButton = document.querySelector<HTMLButtonElement>('#restart-button');
const shareButton = document.querySelector<HTMLButtonElement>('#share-button');
const helpButton = document.querySelector<HTMLButtonElement>('#help-button');
const helpOverlay = document.querySelector<HTMLElement>('#help-overlay');
const helpPlayButton = document.querySelector<HTMLButtonElement>('#help-play-button');
const posterLink = document.querySelector<HTMLAnchorElement>('#poster-link');
const pulseControls = document.querySelector<HTMLElement>('#pulse-controls');
const pulseUndoButton = document.querySelector<HTMLButtonElement>('#pulse-undo-button');
const pulseClearButton = document.querySelector<HTMLButtonElement>('#pulse-clear-button');
```

```
const pulsePlayButton = document.querySelector<HTMLButtonElement>('#pulse-play-button');

if (
  !root ||
  !restartButton ||
  !shareButton ||
  !helpButton ||
  !helpOverlay ||
  !helpPlayButton ||
  !posterLink ||
  !pulseControls ||
  !pulseUndoButton ||
  !pulseClearButton ||
  !pulsePlayButton
) {
  throw new Error('Event Horizon shell is missing required DOM nodes.');
}

const params = new URLSearchParams(window.location.search);
const mode = params.get('mode') === 'legacy' ? 'legacy' : 'pulse-chain';
const debugInput = params.get('debugInput') === '1';
const seed = params.get('seed') ?? getDailyPulseSeed();

const game: EventHorizonRuntime =
  mode === 'legacy'
    ? new EventHorizonGame(root, {
        seed: 'eh-2026-05-29-alpha',
        startedAt: 1780051200000
      })
    : new PulseMode(root, {
        seed,
        startedAt: Date.now()
      });

await game.start();
game.setInputDebug(debugInput);
pulseControls.hidden = mode === 'legacy';
let pulsePaused = false;

const helpKey = mode === 'legacy' ? 'eventHorizon.helpSeen' : 'eventHorizon.pulseHelpSeen';

const hasSeenHelp = (): boolean => {
  try {
    return localStorage.getItem(helpKey) === '1';
  } catch {
    return false;
  }
};

const markHelpSeen = (): void => {
  try {
    localStorage.setItem(helpKey, '1');
  } catch {
    // localStorage can be unavailable in locked-down browser modes.
  }
};

const openHelp = (): void => {
  helpOverlay.hidden = false;
  helpPlayButton.textContent = 'PLAY';
  game.setPaused(true);
};

const closeHelp = (): void => {
  helpOverlay.hidden = true;
  markHelpSeen();
  game.setPaused(false);
};

if (!hasSeenHelp()) {
  openHelp();
}

restartButton.addEventListener('click', () => {
  posterLink.removeAttribute('href');
  pulsePaused = false;
  game.restart();
  updatePulseControls();
});

helpButton.addEventListener('click', openHelp);
helpPlayButton.addEventListener('click', closeHelp);

shareButton.addEventListener('click', async () => {
  const poster = await game.exportPoster();
  posterLink.href = poster;
});

if (game instanceof PulseMode) {
  pulseUndoButton.addEventListener('click', () => {
    const snapshot = game.getSnapshot();
    if (snapshot.phase === 'build') {
      game.undo();
    } else if (snapshot.phase === 'pulse') {
      pulsePaused = !pulsePaused;
      game.setPaused(pulsePaused);
    } else {
      pulsePaused = false;
      game.restart();
    }
    updatePulseControls();
  });
  pulseClearButton.addEventListener('click', () => {
    const snapshot = game.getSnapshot();
    if (snapshot.phase === 'build') {
      game.clearLinks();
    } else {
      pulsePaused = false;
      game.restart();
    }
    updatePulseControls();
  });
  pulsePlayButton.addEventListener('click', () => {
    if (game.getSnapshot().phase === 'build') {
      game.playPulse();
      pulsePaused = false;
      updatePulseControls();
    }
  });
  window.setInterval(updatePulseControls, 250);
  updatePulseControls();
} else {
  pulseControls.hidden = true;
}

function updatePulseControls(): void {
  if (!(game instanceof PulseMode)) {
    return;
  }
  const controls = pulseControls;
  const undoButton = pulseUndoButton;
  const clearButton = pulseClearButton;
  const playButton = pulsePlayButton;
```

```
  if (!controls || !undoButton || !clearButton || !playButton) {
    return;
  }
  const snapshot = game.getSnapshot();
  controls.dataset.phase = snapshot.phase;
  if (snapshot.phase === 'build') {
    undoButton.hidden = false;
    clearButton.hidden = false;
    playButton.hidden = false;
    undoButton.textContent = 'Undo';
    clearButton.textContent = 'Clear';
    playButton.textContent = 'Play';
    playButton.disabled = snapshot.linksUsed === 0;
    return;
  }
  if (snapshot.phase === 'pulse') {
    undoButton.hidden = false;
    clearButton.hidden = false;
    playButton.hidden = true;
    undoButton.textContent = pulsePaused ? 'Resume' : 'Pause';
    clearButton.textContent = 'Restart';
    playButton.disabled = true;
    return;
  }
  undoButton.hidden = false;
  clearButton.hidden = false;
  playButton.hidden = true;
  undoButton.textContent = 'Replay';
  clearButton.textContent = 'Restart';
  playButton.disabled = true;
}

declare global {
  interface Window {
    __EVENT_HORIZON__?: {
      exportPoster: () => Promise<string>;
      forceEnd: () => void;
      getReplayPayload: () => unknown;
      getSnapshot: () => unknown;
      restart: () => void;
    };
    __EVENT_HORIZON_DEBUG__?: {
      addLink: (fromId: number, toId: number) => unknown;
      clearLinks: () => unknown;
      forceBuildPhase: () => void;
      forceCollapse: () => void;
      forceHelp: (open: boolean) => void;
      forcePulsePhase: () => void;
      getLastInputResult: () => unknown;
      getLinks: () => unknown;
      getMode: () => string;
      getNodes: () => unknown;
      getPulses: () => unknown;
      getReplayPayload: () => unknown;
      getSnapshot: () => unknown;
      playPulse: () => unknown;
      setInputDebug: (enabled: boolean) => void;
      simulateLens: (points: { x: number; y: number }[]) => unknown;
    };
  }
}

window.__EVENT_HORIZON__ = {
  exportPoster: () => game.exportPoster(),
  forceEnd: () => game.forceEnd(),
  getReplayPayload: () => game.getReplayPayload(),
  getSnapshot: () => game.getSnapshot(),
  restart: () => game.restart()
};

window.__EVENT_HORIZON_DEBUG__ =
  game instanceof PulseMode
    ? {
        addLink: (fromId, toId) => game.addLink(fromId, toId),
        clearLinks: () => game.clearLinks(),
        forceBuildPhase: () => game.forceBuildPhase(),
        forceCollapse: () => game.forceCollapse(),
        forceHelp: (open) => {
          if (open) {
            openHelp();
          } else {
            closeHelp();
          }
        },
        forcePulsePhase: () => game.forcePulsePhase(),
        getLastInputResult: () => game.getLastInputResult(),
        getLinks: () => game.getLinks(),
        getMode: () => game.getMode(),
        getNodes: () => game.getNodes(),
        getPulses: () => game.getPulses(),
        getReplayPayload: () => game.getReplayPayload(),
        getSnapshot: () => game.getSnapshot(),
        playPulse: () => game.playPulse(),
        setInputDebug: (enabled) => game.setInputDebug(enabled),
        simulateLens: (points) => game.simulateLens(points)
      }
    : {
        addLink: () => undefined,
        clearLinks: () => undefined,
        forceBuildPhase: () => undefined,
        forceCollapse: () => game.forceEnd(),
        forceHelp: (open) => {
          if (open) {
            openHelp();
          } else {
            closeHelp();
          }
        },
        forcePulsePhase: () => undefined,
        getLastInputResult: () => undefined,
        getLinks: () => [],
        getMode: () => 'legacy',
        getNodes: () => [],
        getPulses: () => [],
        getReplayPayload: () => game.getReplayPayload(),
        getSnapshot: () => game.getSnapshot(),
        playPulse: () => undefined,
        setInputDebug: (enabled) => game.setInputDebug(enabled),
        simulateLens: () => undefined
      };
```

src/styles.css

```
:root {
  color-scheme: dark;
  font-family:
    Inter, ui-sans-serif, system-ui, -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
  background: #03040a;
  color: #f7fbff;
  font-synthesis: none;
```

```
  text-rendering: optimizeLegibility;
  -webkit-font-smoothing: antialiased;
}

* {
  box-sizing: border-box;
  -webkit-tap-highlight-color: transparent;
}

html,
body,
#app,
#game-shell,
#game-root {
  width: 100%;
  height: 100%;
  margin: 0;
  overflow: hidden;
  overscroll-behavior: none;
  touch-action: none;
  user-select: none;
  -webkit-user-select: none;
  -webkit-touch-callout: none;
}

body {
  position: fixed;
  inset: 0;
  background:
    radial-gradient(circle at 50% 42%, rgba(22, 49, 89, 0.24), transparent 38%),
    linear-gradient(180deg, #070912 0%, #03040a 100%);
}

button,
a {
  -webkit-tap-highlight-color: transparent;
}

#game-shell {
  position: fixed;
  inset: 0;
  display: grid;
  place-items: center;
  background: #03040a;
}

#game-root {
  position: absolute;
  inset: 0;
  overflow: hidden;
}

#game-root canvas {
  display: block;
  width: 100%;
  height: 100%;
  touch-action: none;
  user-select: none;
  -webkit-user-select: none;
  -webkit-touch-callout: none;
}

#status-layer {
  position: fixed;
  top: max(12px, env(safe-area-inset-top));
  right: max(12px, env(safe-area-inset-right));
  z-index: 2;
  display: flex;
  gap: 8px;
}

#pulse-controls {
  position: fixed;
  right: max(14px, env(safe-area-inset-right));
  bottom: max(22px, env(safe-area-inset-bottom));
  left: max(14px, env(safe-area-inset-left));
  z-index: 3;
  display: grid;
  grid-template-columns: 1fr 1fr 1.45fr;
  gap: 10px;
  pointer-events: auto;
}

#pulse-controls[hidden] {
  display: none;
}

#pulse-controls[data-phase="pulse"],
#pulse-controls[data-phase="ended"] {
  grid-template-columns: 1fr 1fr;
}

#restart-button,
#share-button,
#help-button,
#poster-link,
#pulse-controls button {
  display: grid;
  place-items: center;
  border: 1px solid rgba(179, 226, 255, 0.26);
  border-radius: 8px;
  background: rgba(7, 12, 25, 0.72);
  color: #f7fbff;
  font-size: 21px;
  line-height: 1;
  text-decoration: none;
  backdrop-filter: blur(8px);
  touch-action: manipulation;
}

#restart-button,
#share-button,
#help-button,
#poster-link {
  width: 40px;
  height: 40px;
}

#pulse-controls button {
  min-height: 48px;
  color: #f7fbff;
  font-size: 14px;
  font-weight: 900;
  text-transform: uppercase;
}

#pulse-controls button[hidden] {
  display: none;
}

#pulse-play-button {
  background: linear-gradient(90deg, rgba(103, 244, 255, 0.9), rgba(210, 103, 255, 0.9));
  color: #061120;
```

```
}

#poster-link:not([href]) {
  display: none;
}

#help-overlay {
  position: fixed;
  inset: 0;
  z-index: 5;
  display: grid;
  place-items: center;
  padding: max(18px, env(safe-area-inset-top)) max(16px, env(safe-area-inset-right))
    max(18px, env(safe-area-inset-bottom)) max(16px, env(safe-area-inset-left));
  background: rgba(2, 4, 10, 0.82);
  backdrop-filter: blur(12px);
  touch-action: none;
}

#help-overlay[hidden] {
  display: none;
}

#help-panel {
  width: min(92vw, 440px);
  max-height: min(88vh, 760px);
  overflow: auto;
  padding: 22px;
  border: 1px solid rgba(139, 222, 255, 0.34);
  border-radius: 8px;
  background: rgba(7, 12, 25, 0.94);
  box-shadow: 0 20px 80px rgba(0, 0, 0, 0.48);
  color: #f7fbff;
}

#help-panel h1 {
  margin: 12px 0 8px;
  font-size: 28px;
  letter-spacing: 0;
}

#help-panel p {
  margin: 10px 0;
  color: #cfefff;
  font-size: 16px;
  line-height: 1.45;
}

#help-panel ol {
  margin: 14px 0;
  padding-left: 22px;
  color: #f2fbff;
  font-size: 15px;
  line-height: 1.45;
}

#help-panel li {
  margin: 8px 0;
}

#help-play-button {
  width: 100%;
  min-height: 52px;
  margin-top: 12px;
  border: 0;
  border-radius: 8px;
  background: linear-gradient(90deg, #67f4ff, #d267ff);
  color: #061120;
  font-size: 17px;
  font-weight: 900;
  letter-spacing: 0;
  touch-action: manipulation;
}

.help-example {
  position: relative;
  height: 126px;
  margin: 2px 0 10px;
  overflow: hidden;
  border-radius: 8px;
  background:
    radial-gradient(circle at 50% 44%, rgba(82, 28, 120, 0.28), transparent 34%),
    radial-gradient(circle at 72% 58%, rgba(103, 244, 255, 0.14), transparent 18%),
    #03040a;
}

.help-hole,
.help-orb,
.help-tether,
.help-swipe {
  position: absolute;
  display: block;
}

.help-hole {
  left: 44%;
  top: 38%;
  width: 48px;
  height: 48px;
  border: 5px solid rgba(103, 244, 255, 0.34);
  border-radius: 50%;
  background: #000;
  box-shadow: 0 0 34px rgba(103, 244, 255, 0.22);
}

.help-orb {
  left: 70%;
  top: 58%;
  width: 30px;
  height: 30px;
  border-radius: 50%;
  background: #d9fbff;
  box-shadow: 0 0 20px #67f4ff, 0 0 46px rgba(210, 103, 255, 0.7);
}

.help-tether {
  left: 52%;
  top: 54%;
  width: 105px;
  height: 4px;
  border-radius: 999px;
  background: #67f4ff;
  box-shadow: 0 0 18px #67f4ff;
  transform: rotate(18deg);
  transform-origin: left center;
}

.help-swipe {
  left: 58%;
  top: 42%;
  width: 104px;
  height: 42px;
  border-bottom: 7px solid #ffffff;
```

```
  border-left: 7px solid transparent;
  border-radius: 0 0 80px 80px;
  box-shadow: 0 12px 20px rgba(210, 103, 255, 0.5);
  transform: rotate(-22deg);
}

@media (min-width: 820px) {
  #game-root {
    inset: 0;
  }
}
```

tests/e2e/playable.spec.ts

```
import { expect, test, type Page } from '@playwright/test';

const WORLD_WIDTH = 1080;
const WORLD_HEIGHT = 1920;

test('help opens on first visit', async ({ page }) => {
  await openGame(page);
  await expect(page.locator('#help-overlay')).toBeVisible();
  await expect(page.locator('#help-title')).toHaveText('EVENT HORIZON');
  await expect(page.locator('#help-overlay')).toContainText('Build a dark-energy chain');
});

test('connect two nodes by tap-tap', async ({ page }) => {
  await openGameAndPlay(page);
  const nodes = await getNodes(page);
  const source = nodes.find((node) => node.type === 'source');
  const target = nodes.find((node) => node.type === 'energy');
  expect(source).toBeTruthy();
  expect(target).toBeTruthy();
  await tapWorld(page, source!.x, source!.y);
  await tapWorld(page, target!.x, target!.y);
  const links = await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.getLinks()) as Array<{ fromId: number; toId: number }>;
  expect(links.some((link) => link.fromId === source!.id && link.toId === target!.id)).toBe(true);
});

test('connect two nodes by drag', async ({ page }) => {
  await openGameAndPlay(page);
  const nodes = await getNodes(page);
  const from = nodes.find((node) => node.type === 'energy');
  const to = nodes.find((node) => node.type === 'delay');
  expect(from).toBeTruthy();
  expect(to).toBeTruthy();
  const a = await worldToScreen(page, from!.x, from!.y);
  const b = await worldToScreen(page, to!.x, to!.y);
  await page.mouse.move(a.x, a.y);
  await page.mouse.down();
  await page.mouse.move(b.x, b.y, { steps: 8 });
  await page.mouse.up();
  const result = await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.getLastInputResult()) as { ok: boolean; message: string };
  expect(result.ok).toBe(true);
});

test('press play, pulse moves, and energy node scores', async ({ page }) => {
  await openGameAndPlay(page);
  await buildTutorialChain(page);
  await page.locator('#pulse-play-button').click();
  await page.waitForFunction(() => {
    const snapshot = window.__EVENT_HORIZON__?.getSnapshot() as { phase?: string; pulses?: unknown[] };
    return snapshot?.phase === 'pulse' && Number(snapshot.pulses?.length) > 0;
  });
  const before = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot()) as { score: number };
  await page.waitForFunction((score) => {
    const snapshot = window.__EVENT_HORIZON__?.getSnapshot() as { score?: number };
    return Number(snapshot?.score) > Number(score);
  }, before.score);
  const after = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot()) as { score: number; multiplier: number; phase: string };
  expect(after.phase).toBe('pulse');
  expect(after.score).toBeGreaterThan(before.score);
  expect(after.multiplier).toBeGreaterThanOrEqual(1);
});

test('swipe during pulse phase creates Horizon Lens and records replay inputs', async ({ page }) => {
  await openGameAndPlay(page);
  await buildTutorialChain(page);
  await page.locator('#pulse-play-button').click();
  await page.waitForFunction(() => (window.__EVENT_HORIZON__?.getSnapshot() as { phase?: string })?.phase === 'pulse');
  const nodes = await getNodes(page);
  const a = nodes.find((node) => node.id === 6) ?? nodes[4];
  const b = nodes.find((node) => node.id === 8) ?? nodes[5];
  const start = await worldToScreen(page, a.x, a.y);
  const end = await worldToScreen(page, b.x, b.y);
  await page.mouse.move(start.x, start.y);
  await page.mouse.down();
  await page.mouse.move((start.x + end.x) / 2, (start.y + end.y) / 2 - 30, { steps: 5 });
  await page.mouse.move(end.x, end.y, { steps: 5 });
  await page.mouse.up();
  await page.waitForTimeout(120);
  const result = await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.getLastInputResult()) as { kind: string; message: string };
  const replay = await page.evaluate(() => window.__EVENT_HORIZON__?.getReplayPayload()) as {
    buildInputs: unknown[];
    liveInputs: Array<{ kind: string; success: boolean }>;
  };
  expect(result.kind).toBe('lens');
  expect(['BRIDGE CREATED', 'NO ANCHOR']).toContain(result.message);
  expect(replay.buildInputs.some((input) => (input as { kind: string }).kind === 'play')).toBe(true);
  expect(replay.liveInputs.some((input) => input.kind === 'lens')).toBe(true);
});

test('collapse or stabilized end state is reachable', async ({ page }) => {
  await openGameAndPlay(page);
  await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.forceCollapse());
  await page.waitForTimeout(150);
  const snapshot = await page.evaluate(() => window.__EVENT_HORIZON__?.getSnapshot()) as {
    phase: string;
    collapsed: boolean;
    stabilized: boolean;
  };
  expect(snapshot.phase).toBe('ended');
  expect(snapshot.collapsed || snapshot.stabilized).toBe(true);
});

async function openGame(page: Page): Promise<void> {
  await page.addInitScript(() => {
    window.localStorage.clear();
  });
  await page.goto('./?seed=tutorial&debugInput=1');
  await page.locator('canvas').waitFor({ state: 'visible' });
}

async function openGameAndPlay(page: Page): Promise<void> {
  await openGame(page);
  await page.locator('#help-play-button').click();
  await expect(page.locator('#help-overlay')).toBeHidden();
}

async function buildTutorialChain(page: Page): Promise<void> {
```

```
  const nodes = await getNodes(page);
  const byId = new Map(nodes.map((node) => [node.id, node]));
  for (const [fromId, toId] of [
    [1, 2],
    [2, 3],
    [3, 4],
    [4, 5],
    [4, 6]
  ]) {
    const from = byId.get(fromId);
    const to = byId.get(toId);
    expect(from).toBeTruthy();
    expect(to).toBeTruthy();
    await tapWorld(page, from!.x, from!.y);
    await tapWorld(page, to!.x, to!.y);
  }
}

async function getNodes(page: Page) {
  return (await page.evaluate(() => window.__EVENT_HORIZON_DEBUG__?.getNodes())) as Array<{
    id: number;
    type: string;
    x: number;
    y: number;
  }>;
}

async function tapWorld(page: Page, x: number, y: number): Promise<void> {
  const point = await worldToScreen(page, x, y);
  await page.mouse.click(point.x, point.y);
}

async function worldToScreen(page: Page, x: number, y: number): Promise<{ x: number; y: number }> {
  const box = await page.locator('canvas').boundingBox();
  if (!box) {
    throw new Error('Canvas box unavailable.');
  }
  const scale = Math.min(box.width / WORLD_WIDTH, box.height / WORLD_HEIGHT);
  const offsetX = (box.width - WORLD_WIDTH * scale) / 2;
  const offsetY = (box.height - WORLD_HEIGHT * scale) / 2;
  return {
    x: box.x + offsetX + x * scale,
    y: box.y + offsetY + y * scale
  };
}
```

tests/pulse-simulation.test.ts

```
import { describe, expect, it } from 'vitest';
import { generatePulseLevel } from '../src/game/pulse/PulseLevelGenerator';
import { PulseSimulation } from '../src/game/pulse/PulseSimulation';

const options = {
  seed: 'tutorial',
  startedAt: 1780185600000
};

describe('pulse-chain mode', () => {
  it('seeded level generation is deterministic', () => {
    expect(generatePulseLevel('abc')).toEqual(generatePulseLevel('abc'));
    expect(generatePulseLevel('abc').nodes).not.toEqual(generatePulseLevel('xyz').nodes);
  });

  it('same seed generates same nodes', () => {
    const first = new PulseSimulation(options).getNodes();
    const second = new PulseSimulation(options).getNodes();
    expect(second).toEqual(first);
  });

  it('link placement respects budget and rejects duplicate/self links', () => {
    const sim = new PulseSimulation(options);
    expect(sim.addLink(1, 1).ok).toBe(false);
    expect(sim.addLink(1, 2).ok).toBe(true);
    expect(sim.addLink(1, 2).ok).toBe(false);
    expect(sim.addLink(1, 3).ok).toBe(true);
    expect(sim.addLink(2, 3).ok).toBe(true);
    expect(sim.addLink(3, 4).ok).toBe(true);
    expect(sim.addLink(4, 5).ok).toBe(true);
    expect(sim.addLink(4, 6).ok).toBe(true);
    expect(sim.addLink(6, 8).ok).toBe(false);
    expect(sim.getSnapshot().linksUsed).toBe(6);
  });

  it('pulse travels from source to connected energy node and increases score/energy', () => {
    const sim = new PulseSimulation(options);
    sim.addLink(1, 2);
    const beforeEnergy = sim.getSnapshot().darkEnergy;
    sim.playPulse();
    step(sim, 1500);
    const snapshot = sim.getSnapshot();
    expect(snapshot.score).toBeGreaterThan(0);
    expect(snapshot.darkEnergy).toBeGreaterThan(beforeEnergy - 1);
  });

  it('delay node pauses pulse briefly', () => {
    const sim = new PulseSimulation(options);
    sim.addLink(1, 2);
    sim.addLink(2, 3);
    sim.playPulse();
    step(sim, 1300);
    const pulse = sim.getPulses()[0];
    expect(pulse?.currentNodeId).toBe(3);
    expect(pulse?.delayMs).toBeGreaterThan(0);
  });

  it('splitter creates child pulses', () => {
    const sim = new PulseSimulation(options);
    sim.addLink(1, 2);
    sim.addLink(2, 3);
    sim.addLink(3, 4);
    sim.addLink(4, 5);
    sim.addLink(4, 6);
    sim.playPulse();
    step(sim, 2700);
    expect(sim.getPulses().length).toBeGreaterThanOrEqual(2);
  });

  it('long loop increases multiplier', () => {
    const sim = new PulseSimulation(options);
    sim.addLink(1, 2);
    sim.addLink(2, 3);
    sim.addLink(3, 4);
    sim.addLink(4, 7);
    sim.addLink(7, 9);
    sim.addLink(9, 2);
    sim.playPulse();
    step(sim, 9000);
    const snapshot = sim.getSnapshot();
    expect(snapshot.loopsCompleted).toBeGreaterThanOrEqual(1);
    expect(snapshot.maxMultiplier).toBeGreaterThan(1);
```

```
  });

  it('dead end kills pulse', () => {
    const sim = new PulseSimulation(options);
    sim.addLink(1, 2);
    sim.playPulse();
    step(sim, 3600);
    expect(sim.getSnapshot().phase).toBe('ended');
    expect(sim.getSnapshot().endReason).toBe('pulse-died');
  });

  it('Horizon Lens creates a temporary bridge', () => {
    const sim = new PulseSimulation(options);
    sim.addLink(1, 2);
    sim.playPulse();
    const result = sim.applyLens([
      { x: 825, y: 1215, t: 0 },
      { x: 700, y: 1410, t: 90 }
    ]);
    expect(result.ok).toBe(true);
    expect(sim.getLinks().some((link) => link.temporary)).toBe(true);
    expect(sim.getReplayPayload().liveInputs).toHaveLength(1);
  });

  it('replay with same seed and inputs reproduces result and stepHash', () => {
    const first = runScripted();
    const second = runScripted();
    expect(second.getReplayPayload().result).toEqual(first.getReplayPayload().result);
    expect(second.getReplayPayload().stepHash).toBe(first.getReplayPayload().stepHash);
  });
});

function step(sim: PulseSimulation, ms: number): void {
  const frames = Math.ceil(ms / (1000 / 60));
  for (let index = 0; index < frames; index += 1) {
    sim.step(1000 / 60);
  }
}

function runScripted(): PulseSimulation {
  const sim = new PulseSimulation(options);
  for (const [from, to] of [
    [1, 2],
    [2, 3],
    [3, 4],
    [4, 5],
    [4, 6]
  ]) {
    sim.addLink(from, to);
  }
  sim.playPulse();
  sim.applyLens([
    { x: 835, y: 1215, t: 0 },
    { x: 700, y: 1410, t: 120 }
  ]);
  step(sim, 6500);
  return sim;
}
```

tests/score-submit.test.ts

```
import { describe, expect, it } from 'vitest';
import scoreSubmit from '../netlify/functions/score-submit.mjs';

const replay = {
  version: 1,
  seed: 'eh-2026-05-29-alpha',
  startedAt: 1780051200000,
  survivalMs: 67421,
  score: 1280,
  energyCaptured: 87,
  maxStreak: 24,
  tapEvents: [],
  swipeEvents: [],
  phaseTransitions: []
};

const pulseReplay = {
  version: 1,
  mode: 'pulse-chain',
  seed: 'tutorial',
  startedAt: 1780185600000,
  buildInputs: [
    { t: 0, kind: 'link', fromId: 1, toId: 2 },
    { t: 220, kind: 'link', fromId: 2, toId: 3 },
    { t: 520, kind: 'play' }
  ],
  liveInputs: [
    {
      t: 1420,
      kind: 'lens',
      path: [
        { x: 835, y: 1215, t: 0 },
        { x: 700, y: 1410, t: 110 }
      ],
      fromId: 6,
      toId: 8,
      success: true
    }
  ],
  result: {
    score: 620,
    survivalMs: 12120,
    maxMultiplier: 2,
    loopsCompleted: 1,
    linksUsed: 5,
    stabilized: false,
    collapsed: false
  },
  stepHash: '04ce9b1a'
};

describe('score-submit function', () => {
  it('accepts a valid replay payload', async () => {
    const response = await scoreSubmit(
      new Request('https://example.test/.netlify/functions/score-submit', {
        method: 'POST',
        headers: { 'content-type': 'application/json' },
        body: JSON.stringify(replay)
      })
    );
    expect(response.status).toBe(200);
    await expect(response.json()).resolves.toMatchObject({ ok: true, score: 1280 });
  });

  it('accepts a valid pulse-chain replay payload', async () => {
    const response = await scoreSubmit(
      new Request('https://example.test/.netlify/functions/score-submit', {
        method: 'POST',
        headers: { 'content-type': 'application/json' },
```

```
        body: JSON.stringify(pulseReplay)
      })
    );
    expect(response.status).toBe(200);
    await expect(response.json()).resolves.toMatchObject({ ok: true, score: 620, survivalMs: 12120 });
  });

  it('rejects malformed replay payloads', async () => {
    const response = await scoreSubmit(
      new Request('https://example.test/.netlify/functions/score-submit', {
        method: 'POST',
        headers: { 'content-type': 'application/json' },
        body: JSON.stringify({ ok: false })
      })
    );
    expect(response.status).toBe(422);
  });
});
```